NexusSim Software Manual

Version 0.1.0

A Multi-Physics Computational Mechanics Framework

C++20 / Kokkos

NexusSim Development Team

March 2026

© 2024–2026 NexusSim Development Team. Licensed under the Apache License, Version 2.0.

Contents

1 Document Information **2**

2 Scope and Audience **3**

3 Typographic Conventions **4**

4 Symbol Conventions **5**

4.1 Part I — User Guide . . . 5
4.1.1 Introduction . . . 6
4.1.2 Installation and Build . . . 7
4.1.3 Quick Start Tutorial . . . 10
4.1.4 Architecture Overview . . . 13
4.2 Part II — Core Framework . . . 14
4.2.1 Core Infrastructure . . . 15
4.2.2 Data Structures . . . 20
4.2.3 Input/Output . . . 24
4.3 Part III — Element Library . . . 26
4.3.1 Finite Elements . . . 26
4.4 Part IV — Constitutive Models . . . 32
4.4.1 Material Models . . . 33
4.4.2 Failure Models . . . 44
4.4.3 Equations of State . . . 49
4.4.4 Composite Analysis . . . 52
4.5 Part V — Solvers . . . 54
4.5.1 Explicit Dynamics . . . 54
4.5.2 Implicit Solvers . . . 56
4.5.3 Nonlinear Solution Methods . . . 59
4.6 Part VI — Contact and Constraints . . . 60
4.6.1 Contact Mechanics . . . 61
4.6.2 Rigid Bodies and Constraints . . . 63
4.6.3 Loads and Boundary Conditions . . . 65
4.7 Part VII — Multi-Physics . . . 66

4.7.1 Smoothed Particle Hydrodynamics . . . 66
4.7.2 Peridynamics . . . 67
4.7.3 Multi-Physics Coupling . . . 70
4.7.4 Thermal Analysis . . . 71
4.8 Part VIII — Advanced Features . . . 73
4.8.1 ALE Mesh Management . . . 74
4.8.2 Sensors and Controls . . . 75
4.8.3 Advanced Capabilities (Wave 16) . . . 77
4.8.4 Parallel Computing (Wave 17) . . . 78
4.9 Part IX — Verification and Validation . . . 80
4.9.1 Test Suite . . . 80
4.10 Appendices . . . 84
4.10.1 Appendix A — Complete Header File Index . . . 84
4.10.2 Appendix B — Mathematical Notation . . . 89
4.10.3 Appendix C — Material Parameter Reference . . . 90
4.10.4 Appendix D — Element Reference Cards . . . 91

Version 0.1.0 | **Date** March 2026 | **Framework** C++20 / Kokkos | **License** Apache 2.0

CHAPTER 1

Document Information

Field	Value
Document Title	NexusSim Software Manual
Version	0.1.0
Date	March 2026
Classification	Unclassified / Public
Copyright	2024–2026 NexusSim Development Team
License	Apache License, Version 2.0

CHAPTER 2

Scope and Audience

This manual serves as the authoritative reference for the NexusSim computational mechanics framework. It is intended for the following audiences:

- **Simulation engineers** who use NexusSim to set up and execute analyses.
- **Developers** who extend the framework with new elements, materials, or solvers.
- **Researchers** who require detailed knowledge of the underlying mathematical formulations.

The document assumes familiarity with continuum mechanics, the finite element method, and C++ programming.

CHAPTER 3

Typographic Conventions

Throughout this manual, the following conventions are observed:

Convention	Meaning
`monospace`	Source code, class names, file names, commands
italic	Mathematical variables, emphasis
bold	Key terms upon first introduction
σ_{ij}	Mathematical notation (LaTeX)
"See *Section 5.1* (page 15)"	Cross-reference to another section

CHAPTER 4

Symbol Conventions

Symbol	Meaning
$\boldsymbol{\sigma}$	Cauchy stress tensor
$\boldsymbol{\varepsilon}$	Infinitesimal strain tensor
$\mathbf{F}$	Deformation gradient
$\mathbf{B}$	Left Cauchy–Green deformation tensor ($\mathbf{F}\mathbf{F}^T$)
J	Jacobian determinant ($\det \mathbf{F}$)
E, ν	Young's modulus, Poisson's ratio
λ, μ	Lamé parameters
ρ	Mass density
δ	Peridynamic horizon
h	SPH smoothing length
$\mathbf{K}$	Stiffness matrix
$\mathbf{M}$	Mass matrix
$\mathbf{u}$	Displacement vector
$\mathbf{v}$	Velocity vector
Δt	Time step
N_i	Shape function for node i

4.1 Part I — User Guide

This part introduces the NexusSim framework, describes how to obtain, build, and verify the software, and provides a guided tutorial for first-time users.

4.1.1 Introduction

Purpose

NexusSim is a next-generation computational mechanics framework designed for multi-physics simulations at exascale. It provides a unified platform encompassing the Finite Element Method (FEM), Smoothed Particle Hydrodynamics (SPH), and Peridynamics (PD), with hardware portability achieved through the Kokkos programming model.

The framework is written in C++20 and adopts a header-heavy architecture to facilitate template inlining and GPU kernel generation. It targets both shared-memory parallelism (OpenMP) and GPU acceleration (CUDA, HIP, SYCL) through a single, portable source base.

Capabilities

NexusSim provides the following principal capabilities:

1. **Structural analysis** — Linear and nonlinear static and dynamic analyses using ten element types, fourteen material models, and six failure criteria.
2. **Explicit dynamics** — Central-difference time integration with adaptive time stepping, subcycling, element erosion, and energy monitoring.
3. **Implicit solvers** — Newton–Raphson iteration with conjugate gradient and direct linear solvers, Newmark-β time integration, and arc-length path tracing.
4. **Contact mechanics** — Seven contact formulations including penalty, mortar, Hertzian, and tied contact with failure.
5. **Smoothed Particle Hydrodynamics** — Weakly compressible SPH with multiple kernel functions and neighbor search algorithms.
6. **Peridynamics** — Bond-based, ordinary state-based, and non-ordinary correspondence formulations with specialized ceramic, metal, and geomaterial models.
7. **Multi-physics coupling** — FEM–SPH and FEM–PD coupling via Arlequin blending, mortar integrals, element morphing, and adaptive zone reclassification.
8. **Composite analysis** — Classical Lamination Theory with thermal residual stresses, interlaminar shear, and progressive failure analysis.
9. **ALE mesh management** — Lagrangian–Eulerian mesh smoothing and advection.
10. **Sensors and controls** — Runtime monitoring and simulation steering.

Codebase Metrics

The following table summarizes the current state of the codebase:

Metric	Value
Header files (.hpp)	126
Source files (.cpp)	15
Test files (.cpp)	95
Header lines of code	~46,100
Source lines of code	~4,700
Test lines of code	~36,100
Total lines of code	~86,900
Element types	10
Material models (FEM)	14
Material models (PD)	3
Failure models	6
EOS models	5
Contact formulations	7
Test assertions	1,400+

Technology Stack

Component	Technology
Language	C++20 (ISO/IEC 14882:2020)
GPU portability	Kokkos 3.7
Parallelism	OpenMP (default), CUDA (available via Kokkos rebuild)
Build system	CMake 3.25+
Optional packages	MPI, PETSc, HDF5, Eigen3, spdlog, yaml-cpp

4.1.2 Installation and Build

Prerequisites

The following software is required to build NexusSim:

- A C++20-compliant compiler (GCC 11+, Clang 14+, or MSVC 19.30+)
- CMake 3.25 or later
- Kokkos 3.7 (bundled in the `external/` directory)

The following packages are optional:

Package	Purpose	CMake Option
MPI	Distributed-memory parallelism	`NEXUSSIM_ENABLE_MPI`
PETSc	Scalable linear solvers	`NEXUSSIM_ENABLE_PETSC`
HDF5	High-performance I/O	Detected automatically
Eigen3	Dense linear algebra	Detected automatically
spdlog	Structured logging backend	Detected automatically
yaml-cpp	YAML configuration parsing	Detected automatically
pybind11	Python bindings	`NEXUSSIM_BUILD_PYTHON`

Building from Source

The standard build procedure is as follows:

```
# Configure the build (minimal configuration)
cmake -S . -B build \
  -DNEXUSSIM_ENABLE_MPI=OFF \
  -DNEXUSSIM_BUILD_PYTHON=OFF

# Compile
cmake --build build -j$(nproc)
```

CMake Options

The following table lists all user-configurable CMake options:

Option	Default	Description
`NEXUSSIM_ENABLE_GPU`	`ON`	Enable GPU acceleration via Kokkos
`NEXUSSIM_ENABLE_MPI`	`ON`	Enable MPI distributed parallelism
`NEXUSSIM_BUILD_PYTHON`	`ON`	Build Python bindings via pybind11
`NEXUSSIM_ENABLE_PETSC`	`OFF`	Enable PETSc solver backend

Compile Definitions

The build system defines the following preprocessor symbols when the corresponding packages are detected:

Definition	Condition	Effect
`NEXUSSIM_HAVE_MPI`	MPI found	Enables MPI code paths
`NEXUSSIM_HAVE_KOKKOS`	Kokkos found	Enables GPU code paths
`NEXUSSIM_HAVE_PETSC`	PETSc found	Enables PETSc solvers
`NEXUSSIM_HAVE_HDF5`	HDF5 found	Enables HDF5 I/O
`NEXUSSIM_HAVE_SPDLOG`	spdlog found	Enables spdlog logging backend

Build Verification

After a successful build, the test suite may be executed to verify correctness:

```
# Run a single test
./build/bin/material_models_test

# Run all tests
for test in build/bin/*_test; do
    echo "Running $test..."
    "$test" || echo "FAILED: $test"
done
```

All tests should report zero failures. See *Chapter 25* (page 80) for a detailed description of the test infrastructure.

GPU Build Configuration

The development system is equipped with an NVIDIA GeForce GT 1030 GPU (Pascal architecture, compute capability 6.1, 2 GB GDDR5, 384 CUDA cores). Three Kokkos installations are available:

Installation	Location	Backends	Compiler
System	`/usr/lib/cmake/Kokkos`	OpenMP, Serial	g++
CUDA (system)	`/usr/local/lib/cmake/Kokkos`	CUDA, Serial	nvcc_wrapper
CUDA (bundled)	`external/kokkos/lib/cmake/Kokkos`	CUDA, Serial	nvcc_wrapper

The default build uses the system Kokkos (OpenMP backend). To build with CUDA acceleration using the bundled Kokkos, specify the `nvcc_wrapper` compiler and set `CUDA_ROOT` to match the CUDA toolkit version Kokkos was built with:

```
CUDA_ROOT=/usr/local/cuda-12.6 cmake -S . -B build-cuda \
  -DCMAKE_CXX_COMPILER=$PWD/external/kokkos/bin/nvcc_wrapper \
  -DKokkos_DIR=$PWD/external/kokkos/lib/cmake/Kokkos \
  -DNEXUSSIM_ENABLE_GPU=ON \
  -DNEXUSSIM_ENABLE_MPI=OFF \
  -DNEXUSSIM_BUILD_PYTHON=OFF \
  -DNEXUSSIM_ENABLE_OPENMP=OFF \
  -DCMAKE_CXX_STANDARD=17

CUDA_ROOT=/usr/local/cuda-12.6 cmake --build build-cuda -j$(nproc)
```

When `NEXUSSIM_ENABLE_GPU` is `ON` and a CUDA-enabled Kokkos is found, the build system defines `NEXUSSIM_HAVE_KOKKOS` and links against Kokkos CUDA runtime libraries. All Kokkos-annotated kernels (`KOKKOS_INLINE_FUNCTION`) will then execute on the GPU.

GPU Performance (GT 1030)

The following table summarizes GPU speedup over the 64-thread OpenMP CPU backend on representative benchmarks. The GT 1030 is an entry-level GPU; production compute GPUs would show substantially larger speedups.

Benchmark	Problem Size	CPU (ms)	GPU (ms)	Speedup
Vector Add (DAXPY)	100K	4.32	0.19	22.5x
Element Forces (Hex8)	12.5K elements	9.08	0.18	49.6x
CG Solve (3D Laplacian)	27K DOFs	2955.7	118.5	24.9x
FEM Explicit Step (Hex8)	27K elements	13.53	2.72	5.0x

The GPU excels at compute-bound operations (element forces, CG solver) while memory-bandwidth-bound operations (SpMV at 1M, PD bond forces) favor the CPU on this entry-level hardware. See *Chapter 25* (page 80) for detailed results.

Platform Notes

Linux (GCC 13.3): This is the primary development platform. The default build uses the OpenMP backend for shared-memory parallelism on the host CPU. CUDA acceleration requires the NVIDIA CUDA Toolkit 12.0 or later, a compute-capable GPU device, and a Kokkos installation built with the CUDA backend and `nvcc_wrapper`.

macOS: Supported via Clang. OpenMP support requires `libomp` from Homebrew. CUDA is not supported on macOS.

Windows: Supported via MSVC 2022 or later. The build has been tested with the Visual Studio CMake generator.

4.1.3 Quick Start Tutorial

This chapter provides three minimal working examples that demonstrate the principal capabilities of NexusSim. Each example is self-contained and can be compiled as a standalone executable.

Example 1: Creating a Mesh

The following program creates a simple hexahedral mesh, assigns node coordinates, and writes the mesh to VTK format for visualization.

```
#include <nexussim/data/mesh.hpp>
#include <nexussim/io/vtk_writer.hpp>
#include <iostream>

int main() {
    // Create a mesh with 8 nodes (one hex element)
    nxs::Mesh mesh(8);
```

(continues on next page)

(continued from previous page)

```
    // Define node coordinates for a unit cube
    mesh.set_node_coordinates(0, {0.0, 0.0, 0.0});
    mesh.set_node_coordinates(1, {1.0, 0.0, 0.0});
    mesh.set_node_coordinates(2, {1.0, 1.0, 0.0});
    mesh.set_node_coordinates(3, {0.0, 1.0, 0.0});
    mesh.set_node_coordinates(4, {0.0, 0.0, 1.0});
    mesh.set_node_coordinates(5, {1.0, 0.0, 1.0});
    mesh.set_node_coordinates(6, {1.0, 1.0, 1.0});
    mesh.set_node_coordinates(7, {0.0, 1.0, 1.0});

    // Add one Hex8 element block
    auto block_id = mesh.add_element_block("solid", nxs::ElementType::Hex8, 1,␣
↪8);
    auto& block = mesh.element_block(block_id);
    auto nodes = block.element_nodes(0);
    for (size_t i = 0; i < 8; ++i) nodes[i] = i;

    mesh.print_info();
    return 0;
}
```

Example 2: Linear Static FEM Analysis

The following program performs a linear static analysis of a cantilever beam modeled with hexahedral elements. One end is fully constrained; a point load is applied to the opposite end.

```
#include <nexussim/data/mesh.hpp>
#include <nexussim/solver/fem_static_solver.hpp>
#include <iostream>

int main() {
    // Build a 10-element cantilever beam mesh
    const int nx = 10;
    const int num_nodes = (nx + 1) * 4;  // 4 nodes per cross-section
    nxs::Mesh mesh(num_nodes);

    // ... (assign coordinates and connectivity) ...

    // Create the static solver
    nxs::solver::FEMStaticSolver solver(mesh);
    solver.set_material(210.0e9, 0.3);  // Steel: E = 210 GPa, nu = 0.3

    // Fix the left end (nodes 0-3)
    for (int i = 0; i < 4; ++i)
        solver.fix_node_all(i);
```

(continues on next page)

(continued from previous page)

```
    // Apply downward force at the right end
    solver.add_force(num_nodes - 1, 2, -1000.0);  // Fz = -1000 N

    // Solve
    solver.solve_linear();

    // Report maximum displacement
    std::cout << "Max displacement: " << solver.max_displacement() << " m\n";
    return 0;
}
```

Example 3: Explicit Dynamics with Contact

The following example illustrates how to set up an explicit dynamics simulation with contact detection. The reader is referred to *Chapter 13* (page 54) and *Chapter 16* (page 61) for detailed descriptions of the solver and contact algorithms.

```
#include <nexussim/fem/fem_solver.hpp>
#include <nexussim/fem/contact.hpp>
#include <nexussim/data/state.hpp>

int main() {
    nxs::Mesh mesh(/* ... */);
    nxs::State state(mesh);

    // Configure the explicit solver
    nxs::physics::AdaptiveTimestep timestep;
    Real dt = timestep.compute_stable_dt(mesh, state);

    // Time loop
    for (int step = 0; step < 1000; ++step) {
        // 1. Compute internal forces
        // 2. Detect and enforce contact
        // 3. Time-integrate (central difference)
        state.advance_time(dt);
        state.advance_step();
    }
    return 0;
}
```

4.1.4 Architecture Overview

Directory Structure

The NexusSim source tree is organized as follows:

```
NexusSim/
├── include/nexussim/          126 public headers
│   ├── core/          (7)     Type system, exceptions, GPU, logging, memory, MPI
│   ├── data/          (4)     Mesh, Field, State containers
│   ├── coupling/      (3)     Multi-physics coupling operators
│   ├── discretization/(11)    Element implementations
│   ├── solver/        (6)     Implicit solvers, sparse matrix
│   ├── physics/       (43)    Materials, failure, composites, EOS, time␣
↪integration
│   ├── fem/           (18)    Contact, constraints, loads, sensors, controls
│   ├── io/            (14)    Readers, writers, checkpoint, output
│   ├── sph/           (4)     SPH solver, kernels, coupling
│   ├── peridynamics/  (15)    PD solver, materials, coupling
│   ├── ale/           (1)     ALE mesh management
│   └── utils/         (1)     Performance instrumentation
├── src/                       15 source files
├── examples/                  95 test executables
├── cmake/                     CMake find-modules
└── docs/                      Documentation
```

Design Philosophy

The framework is built on the following design principles:

1. **Header-heavy architecture.** The majority of implementation resides in header files to enable template instantiation and `KOKKOS_INLINE_FUNCTION` GPU kernel generation. Only platform-specific or large translation units are placed in `.cpp` files.
2. **Kokkos portability.** All performance-critical data structures use Kokkos views, and computational kernels are annotated with `KOKKOS_INLINE_FUNCTION` so that the same source code compiles for CPU (OpenMP, Serial) and GPU (CUDA, HIP, SYCL) backends.
3. **Structure-of-Arrays (SOA) layout.** The `Field<T>` container stores each component contiguously (all x-components, then all y-components, etc.) to maximize vectorization efficiency and cache utilization.
4. **Zero-overhead abstractions.** The type system, smart pointers, and container wrappers are designed to compile down to the same machine code as hand-written C equivalents.

Namespace Organization

All NexusSim symbols reside within the `nxs` namespace:

```
nxs::                              Root namespace
├── physics::                      Material models, elements, time integration,␣
```

(continues on next page)

(continued from previous page)

```
↪EOS, ALE
│    ├── MaterialType              Material type enumeration (14 types)
│    ├── ElementType               Element type enumeration (12 types)
│    ├── failure::                 Failure models
│    └── composite::               Composite analysis
├── solver::                       Implicit solvers, sparse matrices
├── coupling::                     Multi-physics coupling operators
├── pd::                           Peridynamics
├── sph::                          Smoothed Particle Hydrodynamics
├── io::                           Input/output subsystem
├── memory::                       Memory alignment and allocation
├── utils::                        Performance instrumentation
└── constants::                    Mathematical and physical constants
```

Design Patterns

The framework employs the following design patterns:

Singleton pattern. Used for global services that require exactly one instance: `Logger`, `Profiler`, `FieldRegistry`, `MPIManager`, `KokkosManager`, `MemoryTracker`.

Factory pattern. `ElementFactory` creates element objects by type; `CouplingOperatorFactory` creates coupling operators by method.

RAII (Resource Acquisition Is Initialization). `KokkosManager` initializes Kokkos in its constructor and finalizes it in its destructor. `ScopedTimer`, `MemoryArena`, and `PETScContext` follow the same pattern.

Callback architecture. The Newton–Raphson and arc-length solvers accept `std::function` callbacks for internal force computation and tangent stiffness assembly, decoupling the solver from the element library.

Data Flow

A typical explicit dynamics simulation follows this data flow:

1. **Mesh creation** — Nodes and element connectivity are defined in the `Mesh` object.
2. **State initialization** — The `State` object allocates default nodal fields (displacement, velocity, acceleration, force, mass).
3. **Material assignment** — `MaterialProperties` and `MaterialState` are assigned to each element.
4. **Time loop** — At each step: a. Element internal forces are computed from current displacements and stresses. b. Contact forces are detected and enforced. c. The time integrator advances the solution (central difference). d. Boundary conditions are applied. e. Output is written at specified intervals.

4.2 Part II — Core Framework

This part describes the foundational infrastructure upon which all NexusSim modules are built: the type system, memory management, GPU abstraction layer, data structures, and input/output subsystem.

4.2.1 Core Infrastructure

Type System

The type system is defined in `core/types.hpp` and establishes the fundamental numeric types used throughout the framework.

Precision Types

Type	C++ Definition	Purpose
`Real`	`double`	Floating-point precision (configurable via `NEXUSSIM_REAL`)
`Index`	`std::size_t`	Array indices and sizes
`Int`	`std::int32_t`	Integer values
`Int64`	`std::int64_t`	Large integer values (step counters)

The precision of `Real` may be changed at compile time by defining the preprocessor symbol `NEXUSSIM_REAL` (for example, `-DNEXUSSIM_REAL=float` for single-precision builds).

Fixed-Size Vector Types

NexusSim provides fixed-size vector aliases based on `std::array`:

Type	Definition	Common Aliases
`Vec2<T>`	`std::array<T, 2>`	`Vec2r`, `Vec2i`
`Vec3<T>`	`std::array<T, 3>`	`Vec3r`, `Vec3i`
`Vec4<T>`	`std::array<T, 4>`	`Vec4r`, `Vec4i`
`Vec6<T>`	`std::array<T, 6>`	`Vec6r` (Voigt tensors)

Enumerations

The following scoped enumerations define the principal categorization types:

`FieldType` — `Scalar`, `Vector`, `Tensor`

`FieldLocation` — `Node`, `Element`, `IntegrationPoint`, `Particle`

`SolverType` — `Explicit`, `Implicit`, `QuasiStatic`

`TimeIntegrator` — `CentralDifference`, `NewmarkBeta`, `GeneralizedAlpha`, `RungeKutta4`

`LinearSolverType` — `Direct`, `ConjugateGradient`, `GMRES`, `BiCGSTAB`

`ExecutionSpace` — `CPU`, `GPU`, `Auto`

Mathematical Constants

The namespace `nxs::constants` provides compile-time mathematical constants as variable templates:

Constant	Value
`pi<T>`	$3.14159265358979\ldots$
`two_pi<T>`	2π
`half_pi<T>`	$\pi/2$
`sqrt_two<T>`	$\sqrt{2}$
`sqrt_three<T>`	$\sqrt{3}$
`epsilon<T>`	10^{-12} (double) or 10^{-6} (float)

Exception System

The exception hierarchy is defined in `core/exception.hpp`. All NexusSim exceptions derive from the base class `nxs::Exception`, which itself inherits from `std::runtime_error`. Each exception automatically captures the source location (file, line, function) via C++20 `std::source_location` when available, or compiler built-ins as a fallback.

Exception Class	Purpose
`Exception`	Base class for all NexusSim exceptions
`LogicError`	Programming errors (assertion failures)
`RuntimeError`	Runtime errors
`InvalidArgumentError`	Invalid function arguments
`OutOfRangeError`	Index out of bounds
`NotImplementedError`	Unimplemented features
`FileIOError`	File operation failures
`ConvergenceError`	Solver convergence failures
`GPUError`	GPU/Kokkos errors
`MPIError`	MPI communication errors

Assertion Macros

The following macros provide guarded assertions that throw specific exception types:

```
NXS_ASSERT(condition, message)        // Throws LogicError
NXS_REQUIRE(condition, message)       // Throws InvalidArgumentError
NXS_CHECK_RANGE(index, size)          // Throws OutOfRangeError
NXS_NOT_IMPLEMENTED()                 // Throws NotImplementedError
NXS_DEBUG_ASSERT(condition, message)  // Active only in debug builds
```

GPU and Kokkos Abstraction Layer

The GPU abstraction layer, defined in `core/gpu.hpp`, provides a portable interface to Kokkos. When Kokkos is not available (`NEXUSSIM_HAVE_KOKKOS` is not defined), all GPU-related types and functions degrade gracefully to no-op stubs.

KokkosManager

The `KokkosManager` is a singleton responsible for Kokkos lifecycle management:

```
class KokkosManager {
public:
    static KokkosManager& instance();
    void initialize(const KokkosConfig& config = KokkosConfig{});
    void finalize();
    bool is_initialized() const;
    void print_configuration() const;
};
```

Parameters. The `KokkosConfig` structure accepts:

- `num_threads` — Number of host threads (`-1` for auto-detection).
- `device_id` — GPU device identifier (default: `0`).
- `disable_warnings` — Suppress Kokkos warnings.
- `tune_internals` — Enable Kokkos internal tuning.

View Type Aliases

Kokkos views are aliased for convenience:

Alias	Kokkos Type
`View1D<T>`	`Kokkos::View<T*, DefaultMemSpace>`
`View2D<T>`	`Kokkos::View<T**, LayoutLeft, DefaultMemSpace>`
`View3D<T>`	`Kokkos::View<T***, LayoutLeft, DefaultMemSpace>`
`HostView1D<T>`	`Kokkos::View<T*, HostMemSpace>`
`HostView2D<T>`	`Kokkos::View<T**, LayoutLeft, HostMemSpace>`

Parallel Execution Wrappers

```
template<typename Functor>
void parallel_for(const std::string& label, std::size_t n, const Functor& f);

template<typename Functor, typename ReduceType>
void parallel_reduce(const std::string& label, std::size_t n,
                     const Functor& f, ReduceType& result);
```

(continues on next page)

(continued from previous page)

```
void fence(const std::string& label = "NexusSim::fence");
```

Logging

The logging subsystem (`core/logger.hpp`) provides a singleton `Logger` with six severity levels: `Trace`, `Debug`, `Info`, `Warn`, `Error`, `Critical`.

Logging macros follow the `fmt` library syntax:

```
NXS_LOG_INFO("Mesh loaded: {} nodes, {} elements", num_nodes, num_elements);
NXS_LOG_WARN("CG solver: slow convergence at iteration {}", iter);
```

When spdlog is available (`NEXUSSIM_HAVE_SPDLOG`), the logger delegates to spdlog; otherwise, it uses a lightweight built-in backend that writes to `stderr`.

Memory Management

The memory subsystem (`core/memory.hpp`) provides four components:

Alignment Utilities

The `nxs::memory` namespace provides aligned allocation functions:

- `DefaultAlignment` — 64 bytes (cache-line alignment).
- `GPUAlignment` — 256 bytes (GPU memory alignment).
- `allocate_aligned(size, alignment)` — Allocates aligned memory.
- `free_aligned(ptr)` — Frees aligned memory.
- `is_aligned(ptr, alignment)` — Tests pointer alignment.

MemoryArena

A fast, stack-like allocator for temporary allocations:

```
class MemoryArena {
public:
    explicit MemoryArena(std::size_t block_size = 1024 * 1024);
    void* allocate(std::size_t size, std::size_t alignment = 64);
    template<typename T, typename... Args>
    T* construct(Args&&... args);
    void reset();
    std::size_t total_allocated() const;
    std::size_t used() const;
};
```

The arena allocates memory in contiguous blocks (default: 1 MB). The `reset()` method rewinds the allocation pointer without freeing memory, making it suitable for per-timestep scratch allocations.

MemoryPool

A fixed-size object pool for rapid allocation and deallocation of objects of a single type:

```
template<typename T, std::size_t ChunkSize = 1024>
class MemoryPool {
public:
    T* allocate();
    void deallocate(T* ptr);
    template<typename... Args>
    T* construct(Args&&... args);
    void destroy(T* ptr);
};
```

AlignedBuffer

An RAII wrapper for cache- or GPU-aligned memory buffers:

```
template<typename T>
class AlignedBuffer {
public:
    explicit AlignedBuffer(std::size_t size, std::size_t alignment = 64);
    T* data();
    std::size_t size() const;
    T& operator[](std::size_t i);
    T& at(std::size_t i);         // Bounds-checked
    void resize(std::size_t new_size);
    void fill(const T& value);
    void zero();
};
```

MemoryTracker

A global singleton that records allocation statistics:

```
class MemoryTracker {
public:
    static MemoryTracker& instance();
    void record_allocation(std::size_t size);
    void record_deallocation(std::size_t size);
    const MemoryStats& stats() const;  // total_allocated, peak_usage, etc.
};
```

MPI Wrapper

The `MPIManager` singleton (`core/mpi.hpp`) provides a simplified interface to MPI collective operations. When MPI is disabled (`NEXUSSIM_HAVE_MPI` is not defined), the manager operates in serial mode (rank 0, size 1) and all collective operations reduce to identity operations.

Supported collective operations include `broadcast`, `reduce_sum`, `allreduce_sum`, `allreduce_max`, and `allreduce_min`.

4.2.2 Data Structures

This chapter describes the three principal data containers: `Field`, `Mesh`, and `State`. These containers form the foundation of all NexusSim simulations.

Field Container

The `Field<T>` class template (`data/field.hpp`) is the fundamental data container in NexusSim. It stores simulation data — scalars, vectors, or tensors — at specified locations (nodes, elements, integration points, or particles).

Storage Layout

Fields employ a **Structure-of-Arrays (SOA)** layout. For a vector field with three components (x, y, z) and n entities, the data are stored as:

$$[\underbrace{x_0, x_1, \ldots, x_{n-1}}_{\text{component 0}}, \underbrace{y_0, y_1, \ldots, y_{n-1}}_{\text{component 1}}, \underbrace{z_0, z_1, \ldots, z_{n-1}}_{\text{component 2}}]$$

This layout maximizes vectorization efficiency and GPU memory coalescing.

Construction

```
Field(std::string name, FieldType type, FieldLocation location,
      std::size_t num_entities, std::size_t num_components = 1);
```

Parameters:

- `name` — Human-readable identifier for the field.
- `type` — One of `Scalar`, `Vector`, or `Tensor`.
- `location` — One of `Node`, `Element`, `IntegrationPoint`, or `Particle`.
- `num_entities` — Number of entities (nodes, elements, etc.).
- `num_components` — Number of components per entity.

Validation: The constructor enforces that scalar fields have exactly 1 component, vector fields have 2–4 components, and tensor fields have 6 (Voigt) or 9 (full) components.

Access Patterns

Scalar access (single-component fields):

```
T& operator[](std::size_t i);
```

Multi-component access:

```
T& at(std::size_t entity_id, std::size_t comp_id);
```

Vector access:

```
template<std::size_t N>
std::array<T, N> get_vec(std::size_t entity_id) const;

template<std::size_t N>
void set_vec(std::size_t entity_id, const std::array<T, N>& vec);
```

Component span (access all values for one component):

```
std::span<T> component(std::size_t comp);
```

Factory Functions

Three convenience functions simplify field creation:

```
Field<T> make_scalar_field(name, location, num_entities);
Field<T> make_vector_field(name, location, num_entities, dim = 3);
Field<T> make_tensor_field(name, location, num_entities, use_voigt = true);
```

Statistical Operations

The `Field` class provides `min()`, `max()`, `sum()`, and `mean()` methods for computing aggregate statistics over all stored values.

Mesh

The `Mesh` class (`data/mesh.hpp`) represents the geometric and topological discretization of the computational domain.

Node Data

The mesh stores node coordinates as a three-component vector field. The following operations are available:

```
std::size_t num_nodes() const;
void set_node_coordinates(std::size_t node_id, const Vec3r& coords);
Vec3r get_node_coordinates(std::size_t node_id) const;
Field<Real>& coordinates();
```

Element Blocks

Elements are organized into **element blocks**, where each block contains a homogeneous group of elements of the same type.

```
struct ElementBlock {
    std::string name;
    ElementType type;
    std::size_t num_nodes_per_elem;
    std::size_t num_elements() const;
    std::span<Index> element_nodes(std::size_t elem_id);
};
```

Adding an element block:

```
Index add_element_block(const std::string& name, ElementType type,
                        std::size_t num_elems, std::size_t nodes_per_elem);
```

Returns: The block identifier, which may be used to retrieve the block via `element_block(block_id)`.

Node Sets and Side Sets

Named collections of nodes and element faces are supported:

```
void add_node_set(const std::string& name, const std::vector<Index>& node_ids);
void add_side_set(const std::string& name,
                  const std::vector<std::pair<Index, Index>>& elem_side_pairs);
```

Bounding Box

```
struct BoundingBox {
    Vec3r min, max;
    Real volume() const;
    Vec3r center() const;
};

BoundingBox compute_bounding_box() const;
```

Simulation State

The `State` class (`data/state.hpp`) manages all time-dependent simulation fields and provides a unified interface for accessing and updating them.

Construction

The `State` object is constructed with a reference to a `Mesh` and automatically allocates five default nodal fields:

Field Name	Type	Components	Description
`displacement`	Vector	3	Nodal displacements
`velocity`	Vector	3	Nodal velocities
`acceleration`	Vector	3	Nodal accelerations
`force`	Vector	3	Nodal force resultants
`mass`	Scalar	1	Lumped nodal masses

Time Management

```
Real time() const;
void advance_time(Real dt);
Int64 step() const;
void advance_step();
```

Field Access

Convenience accessors are provided for standard fields:

```
Field<Real>& displacement();
Field<Real>& velocity();
Field<Real>& acceleration();
Field<Real>& force();
Field<Real>& mass();
Field<Real>& stress();    // Optional - must be added explicitly
Field<Real>& strain();    // Optional - must be added explicitly
```

Additional fields may be registered dynamically:

```
void add_field(const std::string& name, Field<Real>&& field);
bool has_field(const std::string& name) const;
Field<Real>& field(const std::string& name);
```

Energy Computation

```
Real compute_kinetic_energy() const;
// Returns: 0.5 * sum_i(m_i * |v_i|^2)
```

State History

The `StateHistory` class manages multiple time levels for multi-step time integration schemes. It maintains a circular buffer of `State` objects:

```
class StateHistory {
public:
    explicit StateHistory(const Mesh& mesh, std::size_t num_levels = 2);
    State& current();
    State& previous(std::size_t n = 1);
    void advance();
};
```

4.2.3 Input/Output

The I/O subsystem comprises fourteen header files in the `io/` directory, organized into input readers, output writers, and checkpoint/restart facilities.

Input Readers

LS-DYNA Keyword Reader

The `LSDynaReader` class (`io/lsdyna_reader.hpp`) parses LS-DYNA keyword-format (`.k`) files. The reader supports approximately 30 keywords organized into eight groups:

Group	Supported Keywords
Nodes	`*NODE`
Elements	`*ELEMENT_SOLID`, `*ELEMENT_SHELL`, `*ELEMENT_BEAM`
Sections	`*SECTION_SOLID`, `*SECTION_SHELL`, `*SECTION_BEAM`, `*SECTION_SPH`
Materials	`*MAT_ELASTIC`, `*MAT_PLASTIC_KINEMATIC`, `*MAT_JOHNSON_COOK`, and others
Contacts	`*CONTACT_AUTOMATIC_SURFACE_TO_SURFACE`, `*CONTACT_TIED`
Loads	`*LOAD_NODE`, `*LOAD_BODY`, `*BOUNDARY_PRESCRIBED_MOTION`
Curves	`*DEFINE_CURVE`
Constraints	`*CONSTRAINED_NODAL_RIGID_BODY`, `*CONSTRAINED_EXTRA_NODES`

The reader has been validated with 172 test assertions (see *Chapter 25* (page 80)).

Radioss Reader

The `RadiossReader` class (`io/radioss_reader.hpp`) parses Radioss legacy card-based Starter deck format.

Configuration Reader

The `ConfigReader` class (`io/config_reader.hpp`) parses YAML configuration files for simulation parameters and material definitions. This reader requires the yaml-cpp library.

Mesh Validator

The `MeshValidator` class (`io/mesh_validator.hpp`) performs quality checks on imported meshes, including connectivity validation, element orientation checks, and duplicate node detection.

Output Writers

VTK Writer

The `VTKWriter` class (`io/vtk_writer.hpp`) writes simulation results in VTK format for visualization in ParaView or similar tools:

```
class VTKWriter {
public:
    void write_mesh(const std::string& filename, const Mesh& mesh);
    void write_field(const std::string& filename, const Mesh& mesh,
                     const Field<Real>& field);
    void write_time_step(const std::string& base, int step,
                         const Mesh& mesh, const State& state);
};
```

Both legacy (`.vtk`) and XML (`.vtu`) formats are supported.

Animation Writer

The `AnimationWriter` class (`io/animation_writer.hpp`) manages time-series output for creating animation sequences. It writes VTK files at user-specified intervals and generates a ParaView Data (`.pvd`) collection file.

Checkpoint and Restart

Basic Checkpoint

The `CheckpointWriter` and `CheckpointReader` classes (`io/checkpoint.hpp`) provide binary checkpoint I/O:

```
class CheckpointWriter {
    void write(const std::string& filename, const State& state, const Mesh&␣
↪mesh);
};

class CheckpointReader {
    State read(const std::string& filename, const Mesh& mesh);
```

(continues on next page)

(continued from previous page)

```
    bool validate_header(const std::string& filename);
};
```

The binary format consists of a header (magic number, version, endianness indicator) followed by serialized state data.

Extended Checkpoint

The `ExtendedCheckpointWriter` and `ExtendedCheckpointReader` classes (`io/extended_checkpoint.hpp`) extend the basic checkpoint with:

- Material history variables (all 32 per integration point).
- Contact state (active pairs, penetration depths).
- Failure flags (per element).
- Sensor data (complete time histories).

Enhanced Output Modules

Module	Header	Description
Time History	`io/time_history.hpp`	Nodal/element/global results vs. time
Part Energy	`io/part_energy.hpp`	KE, IE, contact energy per part
Interface Force	`io/interface_force_output.hpp`	Contact forces at interfaces
Cross-Section Force	`io/cross_section_force.hpp`	Section cuts: $F_x, F_y, F_z, M_x, M_y, M_z$
Result Database	`io/result_database.hpp`	Binary D3PLOT-like format

4.3 Part III — Element Library

4.3.1 Finite Elements

The NexusSim element library provides seventeen element formulations spanning three-dimensional solids, two-dimensional shells, one-dimensional beams and trusses, discrete spring and damper elements, and extended formulations including thick shells, Kirchhoff elements, and plane/axisymmetric elements. All element implementations reside in the `discretization/` directory and inherit from the abstract base class `nxs::physics::Element`.

Element Base Class

The `Element` class (`physics/element.hpp`) defines the interface that all element types must implement. This design permits the solver to treat all elements polymorphically while retaining the option of static dispatch for performance-critical GPU kernels.

```
class Element {
public:
    struct Properties {
```

(continues on next page)

(continued from previous page)

```
        ElementType type;
        ElementTopology topology;
        int num_nodes;
        int num_gauss_points;
        int num_dof_per_node;
        int spatial_dim;
    };

    virtual Properties properties() const = 0;
    virtual void shape_functions(const Real xi[3], Real* N) const = 0;
    virtual void shape_derivatives(const Real xi[3], Real* dN) const = 0;
    virtual Real jacobian(const Real xi[3], const Real* coords, Real* J) const =␣
↪0;
    virtual void strain_displacement_matrix(const Real xi[3],
                                            const Real* coords, Real* B) const =␣
↪0;
    virtual void gauss_quadrature(Real* points, Real* weights) const = 0;
    virtual void mass_matrix(const Real* coords, Real density, Real* M) const =␣
↪0;
    virtual void stiffness_matrix(const Real* coords, Real E, Real nu,
                                  Real* K) const = 0;
    virtual void internal_force(const Real* coords, const Real* disp,
                                const Real* stress, Real* fint) const = 0;
    virtual Real volume(const Real* coords) const = 0;
    virtual Real characteristic_length(const Real* coords) const = 0;
    virtual bool contains_point(const Real* coords, const Real* point,
                                Real* xi) const = 0;
};
```

All shape function and internal force methods are annotated with `KOKKOS_INLINE_FUNCTION` for GPU portability.

Element Factory

The `ElementFactory` class provides static methods for creating element instances:

```
std::unique_ptr<Element> ElementFactory::create(ElementType type);
ElementType ElementFactory::from_string(const std::string& type_str);
```

Element Summary

The following table summarizes all available element types:

Element	Nodes	DOFs	Gauss Points	Shape Functions	Topology
Hex8	8	24	8 (2^3)	Trilinear	Hexahedron
Hex20	20	60	27 (3^3)	Serendipity quadratic	Hexahedron
Tet4	4	12	1	Linear	Tetrahedron
Tet10	10	30	4	Quadratic	Tetrahedron
Wedge6	6	18	6 (2×3)	Linear prismatic	Wedge
Shell4	4	24	4 (2^2)	Bilinear	Quadrilateral
Shell3	3	18	1 or 3	Linear	Triangle
Beam2	2	12	2	Linear / Hermite	Line
Truss	2	6	1	Linear	Line
Spring/Damper	2	6	N/A	N/A	Discrete
ThickShell8	8	24	8 (2^3)	Trilinear	Hexahedron
ThickShell6	6	18	6 (2×3)	Linear prismatic	Wedge
DKT Shell	3	9	3	Discrete Kirchhoff	Triangle
DKQ Shell	4	12	4 (2^2)	Discrete Kirchhoff	Quadrilateral
Plane	3–4	6–8	1–4	Linear/Bilinear	Tri3/Quad4
Axisymmetric	3–4	6–8	1–4	Linear/Bilinear	Tri3/Quad4
Connector	2	12	N/A	N/A	Discrete

Global Shape Function Derivatives

For all isoparametric solid elements, the physical (global) derivatives of the shape functions are obtained from the natural (parametric) derivatives through the Jacobian transformation:

$$\frac{\partial N_i}{\partial x_j} = \left(\mathbf{J}^{-T}\right)_{jk} \frac{\partial N_i}{\partial \xi_k}$$

where the Jacobian matrix is defined as:

$$J_{ij} = \frac{\partial x_i}{\partial \xi_j} = \sum_{k=1}^{n_{\text{nodes}}} x_k^{(i)} \frac{\partial N_k}{\partial \xi_j}$$

Note

The use of $\mathbf{J}^{-T}$ (the inverse transpose) is essential. An earlier version of the code incorrectly used $\mathbf{J}^{-1}$, which produced correct results only for axis-aligned hexahedral meshes (where $\mathbf{J}$ is diagonal and therefore symmetric). For general meshes — particularly tetrahedral Kuhn decompositions of hexahedral domains — the error ranged from 50% to 700%. This was corrected in all five solid element types (Hex8, Hex20, Tet4, Tet10, Wedge6).

Element Stiffness Matrix

For all continuum elements, the element stiffness matrix is computed by numerical integration:

$$\mathbf{K}_e = \int_{\Omega_e} \mathbf{B}^T \mathbf{D}\, \mathbf{B}\, \mathrm{d}\Omega \approx \sum_{g=1}^{n_{\mathrm{gp}}} \mathbf{B}_g^T\, \mathbf{D}\, \mathbf{B}_g\ \det(\mathbf{J}_g)\, w_g$$

where $\mathbf{B}$ is the strain–displacement matrix, $\mathbf{D}$ is the constitutive matrix, $\mathbf{J}_g$ is the Jacobian at Gauss point g, and w_g is the corresponding quadrature weight.

Hex8 — 8-Node Hexahedron

The Hex8 element is the standard trilinear hexahedral element with eight corner nodes. It is the most commonly used solid element and is suitable for meshes with regular geometry.

Shape functions:

$$N_i(\xi, \eta, \zeta) = \frac{1}{8}(1 + \xi_i \xi)(1 + \eta_i \eta)(1 + \zeta_i \zeta)$$

where $(\xi_i, \eta_i, \zeta_i) \in \{-1, +1\}^3$ are the natural coordinates of node i.

Integration scheme: Full integration with $2 \times 2 \times 2 = 8$ Gauss points.

Key methods:

Method	Description
`shape_functions(xi, N)`	Evaluates 8 shape function values
`shape_derivatives(xi, dN)`	Computes 8×3 parametric derivatives
`shape_derivatives_global(xi, coords, dN_global, detJ)`	Physical derivatives via $\mathbf{J}^{-T}$
`stiffness_matrix(coords, E, nu, K)`	Assembles 24×24 element stiffness
`mass_matrix(coords, rho, M)`	Assembles 24×24 consistent mass
`internal_force(coords, disp, stress, fint)`	Computes 24-component force vector

Known limitations: Susceptible to shear locking for bending-dominated problems in thin structures. Use Hex20 or Shell4 elements in such cases.

Hex20 — 20-Node Serendipity Hexahedron

The Hex20 element extends the Hex8 with mid-edge nodes, providing quadratic interpolation within each element. It is well-suited for problems requiring higher accuracy, such as stress concentrations and curved boundaries.

Nodes: 8 corner nodes + 12 mid-edge nodes = 20 nodes, 60 DOFs.

Integration scheme: Full integration with $3 \times 3 \times 3 = 27$ Gauss points.

Shape functions: Serendipity quadratic functions — corner nodes use the product $(1 + \xi_i\xi)(1 + \eta_i\eta)(1 + \zeta_i\zeta)(\xi_i\xi + \eta_i\eta + \zeta_i\zeta - 2)/8$, while mid-edge nodes use reduced forms.

Tet4 — 4-Node Tetrahedron

The Tet4 is the simplest three-dimensional element. It uses linear shape functions expressed in volume (barycentric) coordinates:

$$N_1 = 1 - \xi - \eta - \zeta, \quad N_2 = \xi, \quad N_3 = \eta, \quad N_4 = \zeta$$

Integration scheme: Single Gauss point at the centroid.

Properties: The Tet4 is a constant-strain element — the strain is uniform throughout the element. This makes it overly stiff for bending problems (volumetric locking) and requires fine meshes for accurate results.

Tet10 — 10-Node Tetrahedron

The Tet10 element adds six mid-edge nodes to the Tet4, yielding quadratic interpolation. It produces significantly more accurate results than the Tet4, particularly for bending-dominated problems.

Nodes: 4 corner + 6 mid-edge = 10 nodes, 30 DOFs.

Integration scheme: 4 Gauss points.

Wedge6 — 6-Node Wedge

The Wedge6 (pentahedral) element has a triangular cross-section with linear interpolation. It arises naturally in transitional regions between hexahedral and tetrahedral meshes.

Nodes: 6 nodes, 18 DOFs.

Integration scheme: $2 \times 3 = 6$ Gauss points (2 through thickness × 3 in triangular cross-section).

Shape functions: Products of triangular area coordinates and linear functions in the prismatic direction.

Shell4 — 4-Node Quadrilateral Shell

The Shell4 element models thin-walled structures with a 6-DOF formulation: three translational DOFs (u, v, w) and three rotational DOFs (θ_x, θ_y, θ_z) per node.

Total DOFs: $4 \times 6 = 24$ per element.

Integration scheme: $2 \times 2 = 4$ Gauss points in the mid-surface.

Stiffness decomposition:

The element stiffness matrix is assembled from three contributions:

$$\mathbf{K}_{\text{local}} = \mathbf{K}_{\text{membrane}} + \mathbf{K}_{\text{bending}} + \mathbf{K}_{\text{shear}}$$

where the membrane stiffness uses the in-plane strain–displacement matrix, the bending stiffness uses the curvature–displacement matrix, and the shear stiffness is based on Reissner–Mindlin theory.

Local-to-global transformation: The element stiffness in global coordinates is obtained as:

$$\mathbf{K}_{\text{global}} = \mathbf{T}^T \mathbf{K}_{\text{local}}\, \mathbf{T}$$

where $\mathbf{T}$ is the 24×24 rotation matrix constructed from the element's local coordinate system.

Known limitation: The Shell4 exhibits shear locking with full integration, producing deflections that are approximately 1.5% of the Euler–Bernoulli beam theory prediction for thin shells. This is a known limitation of the Reissner–Mindlin formulation with full integration and is not a solver defect.

Shell3 — 3-Node Triangular Shell

The Shell3 is a flat triangular shell element with 6 DOFs per node (18 total). It supports 1-point (centroid) or 3-point Gauss integration. The Shell3 arises in meshes generated from surface triangulations.

Beam2 — 2-Node Beam

The Beam2 is a two-node beam element with 6 DOFs per node (12 total): three translations and three rotations. It uses linear interpolation for axial behavior and Hermite cubic interpolation for bending, following Euler–Bernoulli or Timoshenko beam theory.

Integration scheme: 2 Gauss points along the beam axis.

Truss — 2-Node Truss

The Truss element connects two nodes and resists only axial force. Each node has 3 translational DOFs (6 total). A single integration point is used.

Force computation: $F = EA\,\varepsilon$, where A is the cross-sectional area and $\varepsilon = (\ell - \ell_0)/\ell_0$ is the engineering strain.

Spring and Damper Elements

Discrete elements connecting two nodes with 3 translational DOFs per node (6 total). No integration points are used; forces are computed directly from the relative displacement or velocity of the two nodes.

Spring: $F = k\,\Delta x$ (linear stiffness)

Damper: $F = c\,\Delta v$ (viscous damping)

Spring-Damper: $F = k\,\Delta x + c\,\Delta v$ (combined)

All three variants support nonlinear force–displacement or force–velocity curves via the load curve system (see *Section 18.1* (page 65)).

Wave 15: Extended Element Formulations

The following additional elements are available in `physics/elements_wave15.hpp` (namespace `nxs::elements`):

ThickShell8 — 8-node thick shell element with a hex-like topology. Uses full 3D stress evaluation (σ_{xx}, σ_{yy}, σ_{zz}, σ_{xy}, σ_{yz}, σ_{xz}) with $2 \times 2 \times 2$ Gauss integration. Suitable for structures where through-thickness stress cannot be neglected.

ThickShell6 — 6-node thick shell element with a wedge-shaped topology (triangular in-plane, linear through thickness). Uses 2×3 integration. Fills the same role as ThickShell8 for triangular mesh regions.

DKT Shell — Discrete Kirchhoff Triangle with 3 nodes and 9 DOFs (one transverse displacement and two rotations per node). Enforces the Kirchhoff constraint (zero transverse shear) discretely at selected points, avoiding shear locking entirely. Best suited for thin plate and shell bending problems.

DKQ Shell — Discrete Kirchhoff Quadrilateral with 4 nodes and 12 DOFs. Extends the DKT concept to quadrilateral geometry using 2×2 integration. Provides improved accuracy over DKT for regular meshes.

Plane Element — 2D element for plane-stress and plane-strain analysis. Supports both Quad4 (4 nodes, bilinear) and Tri3 (3 nodes, linear) topologies. The stress state flag selects between plane stress ($\sigma_{zz} = 0$) and plane strain ($\varepsilon_{zz} = 0$).

Axisymmetric Element — Models solids of revolution using a 2D cross-section with an additional hoop strain component $\varepsilon_{\theta\theta} = u_r/r$. Supports Quad4 and Tri3 topologies. Integration weights include the $2\pi r$ circumferential factor.

Connector Element — 2-node discrete element with 6 DOFs per node (12 total), representing spot welds, rivets, or bolts. Transmits forces and moments between two nodes with configurable stiffness in all six DOF directions. Supports failure criteria based on normal force, shear force, and moment resultants.

4.4 Part IV — Constitutive Models

This part describes the material models, failure criteria, equations of state, and composite analysis capabilities provided by NexusSim. The constitutive models are implemented in the `physics/` directory and its subdirectories `physics/failure/` and `physics/composite/`.

4.4.1 Material Models

NexusSim provides thirty-four continuum material models for finite element analysis and three additional material models for peridynamics. All FEM material models share a common interface and data structure.

Material Data Structures

MaterialState

The `MaterialState` structure (`physics/material.hpp`) stores the current mechanical state at each integration point:

```
struct MaterialState {
    Real stress[6];                    // Voigt: _xx, _yy, _zz, _xy, _yz, _xz
    Real strain[6];                    // Voigt notation
    Real F[9];                         // Deformation gradient (row-major 3×3)
    Real history[32];                  // Material-specific history variables
    Real effective_plastic_strain;
    Real damage;
    Real temperature;
    bool failed;
};
```

The Voigt notation convention used throughout NexusSim is:

$$\boldsymbol{\sigma} = \begin{bmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{zz} & \sigma_{xy} & \sigma_{yz} & \sigma_{xz} \end{bmatrix}^T$$

$$\boldsymbol{\varepsilon} = \begin{bmatrix} \varepsilon_{xx} & \varepsilon_{yy} & \varepsilon_{zz} & \gamma_{xy} & \gamma_{yz} & \gamma_{xz} \end{bmatrix}^T$$

where $\gamma_{ij} = 2\varepsilon_{ij}$ denotes the engineering shear strains.

MaterialProperties

The `MaterialProperties` structure contains all parameters for all material types. The `type` field selects which parameters are active:

```
struct MaterialProperties {
    physics::MaterialType type;
    Real E, nu, density;              // Basic elastic properties
    Real yield_stress;                // Von Mises / Johnson-Cook
    Real hardening_modulus;           // Isotropic hardening modulus
    // ... extended fields for all material types ...
};
```

Constitutive Update Interface

Each material model implements the following function:

```
void compute_stress(MaterialState& state, const MaterialProperties& props, Real␣
↪dt);
```

Parameters:

- `state` — Current material state; updated in place with the new stress.
- `props` — Material properties (read-only).
- `dt` — Time step (used by rate-dependent models).

Material Model Reference

Elastic (Linear Isotropic)

Parameters: E (Young's modulus), ν (Poisson's ratio)

Constitutive matrix (isotropic):

$$\mathbf{D} = \begin{bmatrix} \lambda + 2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda + 2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda + 2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix}$$

where the Lamé parameters are:

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \qquad \mu = \frac{E}{2(1+\nu)}$$

Application: General structural analysis, benchmark problems.

Von Mises (Elasto-Plastic with Isotropic Hardening)

Parameters: σ_y (initial yield stress), H (hardening modulus)

Yield function:

$$f = \sigma_{\text{vm}} - \sigma_y(\bar{\varepsilon}^p)$$

where the von Mises equivalent stress is:

$$\sigma_{\text{vm}} = \sqrt{\frac{3}{2}\, s_{ij} s_{ij}}$$

and $s_{ij} = \sigma_{ij} - \frac{1}{3}\sigma_{kk}\delta_{ij}$ is the deviatoric stress.

Radial return algorithm:

1. Compute trial stress: $\boldsymbol{\sigma}_{\text{trial}} = \boldsymbol{\sigma}_n + \mathbf{D} : \Delta\boldsymbol{\varepsilon}$
2. Evaluate yield: $f = \sigma_{\text{vm}}(\boldsymbol{\sigma}_{\text{trial}}) - \sigma_y(\bar{\varepsilon}^p)$
3. If $f > 0$ (plastic):

- $\Delta\gamma = f \,/\, (3G + H)$
- $\boldsymbol{\sigma} = \boldsymbol{\sigma}_{\text{trial}} - 3G\,\Delta\gamma\,\mathbf{n}$
- $\bar{\varepsilon}^p \leftarrow \bar{\varepsilon}^p + \Delta\gamma$

where $\mathbf{n}$ is the unit normal to the yield surface.

Application: General metal plasticity.

Johnson-Cook (Viscoplastic)

Parameters: A, B, n, C, m, $\dot{\varepsilon}_0$, T_{melt}, T_{room}

Flow stress:

$$\sigma_y = (A + B\,\bar{\varepsilon}^{p\,n})\left(1 + C\ln\frac{\dot{\varepsilon}}{\dot{\varepsilon}_0}\right)(1 - T^{*m})$$

where the homologous temperature is:

$$T^* = \frac{T - T_{\text{room}}}{T_{\text{melt}} - T_{\text{room}}}$$

Application: High strain-rate metal deformation, impact, and ballistic analyses.

Neo-Hookean (Hyperelastic)

Parameters: $C_{10} = \mu/2$ (where μ is the shear modulus)

Cauchy stress:

$$\boldsymbol{\sigma} = \frac{\mu}{J}(\mathbf{B} - \mathbf{I}) + \frac{\lambda}{J}\ln(J)\,\mathbf{I}$$

where $\mathbf{B} = \mathbf{F}\mathbf{F}^T$ is the left Cauchy–Green tensor and $J = \det(\mathbf{F})$.

Application: Rubber, soft biological tissue, moderate-strain elastomers.

Mooney-Rivlin (Hyperelastic)

Parameters: C_{10}, C_{01}, D_1 (bulk compressibility)

Strain energy density:

$$W = C_{10}(\bar{I}_1 - 3) + C_{01}(\bar{I}_2 - 3) + \frac{1}{D_1}(J - 1)^2$$

where $\bar{I}_1$ and $\bar{I}_2$ are the isochoric invariants of $\mathbf{B}$.

Application: Rubber, elastomers, seals.

Ogden (Hyperelastic)

Parameters: μ_i, α_i for $i = 1, \ldots, N$ (up to 6 terms)

Strain energy density:

$$W = \sum_{i=1}^{N} \frac{2\mu_i}{\alpha_i^2} \left(\bar{\lambda}_1^{\alpha_i} + \bar{\lambda}_2^{\alpha_i} + \bar{\lambda}_3^{\alpha_i} - 3 \right)$$

where $\bar{\lambda}_j$ are the isochoric principal stretches.

Application: Large-stretch rubber behavior with complex stress–strain curves.

Orthotropic (Anisotropic Linear Elastic)

Parameters: E_1, E_2, E_3; G_{12}, G_{13}, G_{23}; $\nu_{12}, \nu_{13}, \nu_{23}$

Constitutive relation: $\boldsymbol{\sigma} = \mathbf{C}\,\boldsymbol{\varepsilon}$ where $\mathbf{C}$ is the 6×6 orthotropic stiffness matrix.

Application: Fiber-reinforced composites, wood, rolled metals.

Piecewise-Linear Plasticity

Parameters: Yield table as pairs $(\bar{\varepsilon}^p, \sigma_y)$

The yield stress is interpolated from a user-supplied table of effective plastic strain versus flow stress. Linear interpolation is used between data points.

Application: General metals with experimentally measured stress–strain curves.

Tabulated Material

Parameters: Stress–strain table with optional strain-rate dependence

A fully general tabulated material that supports stress–strain curves at multiple strain rates. At each time step, the appropriate curve is selected based on the current strain rate, and interpolation provides the flow stress.

Application: Materials characterized by experimental curves.

Foam (Low-Density)

Parameters: E, ν, σ_y (crush plateau), ε_d (densification strain)

Behavior: The foam model implements a three-phase response: elastic, plateau (constant stress), and densification (exponentially increasing stress). Densification occurs when the volumetric strain approaches ε_d:

$$\sigma_d = \sigma_y \cdot \frac{1}{1 - |\varepsilon_v|/\varepsilon_d}$$

The denominator is guarded against values ≤ 0 to prevent numerical overflow.

Application: Polymer foams, packaging, energy absorption.

Crushable Foam

Parameters: Compressive yield stress as a function of volumetric strain $\sigma_p(\varepsilon_v)$, tensile cutoff

The crushable foam model stores the absolute volumetric strain as a positive value in `history[0]`. The compressive yield stress is interpolated from a user-supplied crush curve.

Application: Metallic foams, structural energy absorbers.

Honeycomb (Orthotropic Crush)

Parameters: Normal stress σ_N, transverse stress σ_T, shear stress σ_S, plus nonlinear crush curves for each direction

Application: Honeycomb sandwich cores, crash-box structures.

Viscoelastic (Prony Series)

Parameters: G_∞ (long-term modulus), G_i and β_i for $i = 1, \ldots, N$ (Prony series terms)

The viscoelastic model uses a strain-increment formulation with the previous strain state stored in `history[23-28]`.

Relaxation function:

$$G(t) = G_\infty + \sum_{i=1}^{N} G_i \, e^{-\beta_i t}$$

Application: Polymers, rubber, biological tissue.

Cowper-Symonds (Rate-Dependent Plasticity)

Parameters: σ_y (static yield), D and q (rate parameters)

Dynamic yield stress:

$$\sigma_y^{\text{dyn}} = \sigma_y^{\text{static}} \left[1 + \left(\frac{\dot{\varepsilon}}{D} \right)^{1/q} \right]$$

Application: Steel and aluminum at moderate to high strain rates.

Zhao (Rate-Dependent)

Parameters: A, B, n, C_1, C_2, $\dot{\varepsilon}_0$

Application: Impact problems with complex rate sensitivity.

Elastic-Plastic with Failure

Parameters: σ_y (yield stress), $\varepsilon_{\text{fail}}$ (failure strain)

Combines Von Mises plasticity with a simple maximum plastic strain failure criterion. When $\bar{\varepsilon}^p > \varepsilon_{\text{fail}}$, the element is flagged as failed and its stress is set to zero.

Rigid

Parameters: ρ (density only)

The rigid material has infinite stiffness; the element undergoes no deformation. It is used for impactors, fixtures, and rigid body components.

Null

Parameters: ρ (density only)

The null material carries no stress but contributes mass. It is used for mass-only elements and Lagrangian markers.

Hill Anisotropic Plasticity

Parameters: σ_y (yield stress), R_{00}, R_{45}, R_{90} (Lankford R-values)

Yield function:

$$F(\sigma_{22} - \sigma_{33})^2 + G(\sigma_{33} - \sigma_{11})^2 + H(\sigma_{11} - \sigma_{22})^2 + 2L\sigma_{23}^2 + 2M\sigma_{13}^2 + 2N\sigma_{12}^2 = \sigma_y^2$$

where F, G, H, L, M, N are derived from the R-values.

Application: Sheet metal forming with planar anisotropy.

Barlat Yld2000

Parameters: σ_y, m (exponent), α_1 through α_8 (anisotropy coefficients)

Yield function:

$$\Phi = |S_1' - S_2'|^m + |2S_2'' + S_1''|^m + |2S_1'' + S_2''|^m = 2\sigma_y^m$$

where S_i' and S_i'' are principal values of two linear transformations of the stress tensor, parameterized by the eight α coefficients.

Application: Advanced aluminum sheet forming with strong in-plane anisotropy.

Tabulated Johnson-Cook

Parameters: Strain-rate table $\sigma_y(\bar{\varepsilon}^p, \dot{\varepsilon})$, temperature table $\sigma_y(\bar{\varepsilon}^p, T)$

The flow stress is determined by bilinear interpolation from user-supplied tables of effective plastic strain versus flow stress at multiple strain rates and temperatures, rather than using the analytical Johnson-Cook equation.

Application: Metals with experimentally measured rate and temperature sensitivity.

Concrete (Drucker-Prager Cap)

Parameters: α (friction angle), λ (cap eccentricity), f_c' (compressive strength), f_t (tensile strength)

Yield surface:

$$f_s = \sqrt{J_2} - \alpha\, I_1 - k = 0, \qquad f_c = \sqrt{J_2^2 + (I_1 - L)^2/R^2} - F_c = 0$$

The shear envelope (f_s) is intersected with an elliptical cap (f_c) to capture pressure-dependent hardening and compaction.

Application: Concrete, rock, and cementitious materials under confinement.

Fabric

Parameters: E_1, E_2 (warp/weft moduli), G_{12}, tensile-only flag

The fabric model carries biaxial tension with zero compressive stiffness. Stress is set to zero when the corresponding normal strain is compressive.

Application: Airbags, parachutes, textile membranes.

Cohesive Zone

Parameters: $T_n^{\max}$ (normal traction), $T_s^{\max}$ (shear traction), δ_n^c (normal separation), δ_s^c (shear separation)

Traction-separation law:

$$T_n = T_n^{\max} \frac{\delta_n}{\delta_n^c} \exp\left(1 - \frac{\delta_n}{\delta_n^c}\right)$$

Application: Delamination, adhesive joints, interfacial fracture.

Soil Cap

Parameters: α (Drucker-Prager slope), W (cap hardening parameter), D (cap hardening exponent), R (cap ratio)

A cap plasticity model for geomaterials combining a linear Drucker-Prager shear surface with a hardening elliptical cap. The cap expands with plastic volumetric compaction.

Application: Soils, granular fill, buried structure analyses.

User-Defined Material

Parameters: User-supplied callback function pointer, up to 48 material constants

The user-defined material provides a callback interface for custom constitutive models. The user implements `void user_material(MaterialState&, const Real* params, Real dt)`.

Application: Research materials, proprietary constitutive models.

Arruda-Boyce (8-Chain Rubber)

Parameters: μ (initial shear modulus), λ_m (locking stretch), D (bulk compressibility)

Strain energy density:

$$W = \mu \sum_{i=1}^{5} \frac{C_i}{\lambda_m^{2i-2}} (\bar{I}_1^i - 3^i) + \frac{1}{D}(J-1)^2$$

where C_i are coefficients derived from the inverse Langevin function expansion.

Application: Rubber and elastomers with limited chain extensibility.

Shape Memory Alloy

Parameters: E_A, E_M (austenite/martensite moduli), σ_s^{AM}, σ_f^{AM} (forward transformation stresses), σ_s^{MA}, σ_f^{MA} (reverse), ε_L (max transformation strain)

The model tracks the martensite volume fraction ξ and implements the superelastic loading-unloading hysteresis with full strain recovery upon unloading.

Application: Superelastic NiTi devices, biomedical stents, seismic dampers.

Rate-Dependent Foam

Parameters: Loading curves $\sigma(\varepsilon, \dot{\varepsilon})$, unloading hysteresis factor

Stress-strain response is interpolated from tabulated loading curves at multiple strain rates. Unloading follows a separate hysteretic path scaled by the user-specified factor.

Application: Automotive padding, helmet liners, energy absorbers at varying impact speeds.

Prony Viscoelastic

Parameters: G_∞, K_∞, pairs (g_i, τ_i^G) and (k_i, τ_i^K) for $i = 1, \ldots, N$

Relaxation moduli:

$$G(t) = G_\infty + G_0 \sum_{i=1}^{N} g_i \, e^{-t/\tau_i^G}, \qquad K(t) = K_\infty + K_0 \sum_{i=1}^{N} k_i \, e^{-t/\tau_i^K}$$

Both deviatoric and volumetric relaxation are modeled via independent Prony series branches with recursive convolution integration.

Application: Polymer melts, adhesives, encapsulants with multiple relaxation timescales.

Thermal Elastic-Plastic

Parameters: $E(T)$, $\sigma_y(T)$, $H(T)$, $\alpha_{\text{CTE}}(T)$ — temperature-dependent tables

Material properties are interpolated from temperature-dependent tables at the current element temperature. Thermal strain $\varepsilon^T = \alpha \Delta T$ is subtracted from the total strain before the mechanical constitutive update.

Application: Welding simulation, thermomechanical fatigue, hot forming.

Zerilli-Armstrong

Parameters: C_0, C_1, C_3, C_4, C_5, n (BCC form); C_0, C_2, C_3, C_4 (FCC form)

Flow stress (BCC):

$$\sigma_y = C_0 + C_1 \exp(-C_3 T + C_4 T \ln \dot{\varepsilon}) + C_5 \bar{\varepsilon}^{p\,n}$$

Application: Rate-dependent metals (steel BCC, copper/aluminum FCC) under high strain rate.

Steinberg-Guinan

Parameters: Y_0, $Y_{\max}$, β, n, G_0, G'_P, G'_T, T_m

Flow stress:

$$\sigma_y = Y_0 \left[1 + \beta(\bar{\varepsilon}^p + \varepsilon_i)\right]^n \frac{G(P,T)}{G_0}, \qquad \sigma_y \leq Y_{\max}$$

where $G(P,T)$ accounts for pressure and temperature dependence of the shear modulus.

Application: Metals at very high pressures (>10 GPa) and strain rates (>10^5/s).

MTS (Mechanical Threshold Stress)

Parameters: $\hat{\sigma}_a$, $\hat{\sigma}_i$, $\hat{\sigma}_\varepsilon$, g_{0i}, $g_{0\varepsilon}$, p_i, q_i, A, b, k_b

Flow stress:

$$\sigma_y/G = \sigma_a/G + s_i(\dot{\varepsilon}, T)\,\hat{\sigma}_i/G_0 + s_\varepsilon(\dot{\varepsilon}, T)\,\hat{\sigma}_\varepsilon/G_0$$

where s_i and s_ε are Arrhenius-type scaling functions. The model is grounded in dislocation thermodynamics.

Application: Metals with physically-based rate and temperature dependence (copper, tantalum).

Blatz-Ko with Mullins

Parameters: μ (shear modulus), r (damage parameter), $\eta_{\max}$ (max softening)

Strain energy density:

$$W = \frac{\mu}{2}\left(\frac{I_2}{I_3} + 2\sqrt{I_3} - 5\right)$$

The Mullins effect is modeled via a damage variable that tracks the historical maximum strain energy; subsequent loading below this threshold exhibits softened response.

Application: Filled rubber with stress-softening (Mullins) behavior.

Laminated Glass

Parameters: Glass E_g, ν_g, σ^g_{fail}; PVB interlayer E_{PVB}, ν_{PVB}

A composite material model with brittle glass outer plies and a hyperelastic PVB interlayer. Glass failure is governed by a maximum principal stress criterion; post-failure load is carried by the PVB interlayer and any intact glass plies.

Application: Automotive windshields, architectural glazing, blast-resistant windows.

Spot Weld

Parameters: F_N (normal strength), F_S (shear strength), d (nugget diameter), failure flag

A beam-type element material that transmits forces between connected shell parts. Failure is governed by a combined normal-shear interaction criterion:

$$\left(\frac{f_N}{F_N}\right)^2 + \left(\frac{f_S}{F_S}\right)^2 \geq 1$$

Application: Spot-welded automotive assemblies, joint failure prediction.

Rate-Dependent Composite

Parameters: $E_1(\dot{\varepsilon})$, $E_2(\dot{\varepsilon})$, $G_{12}(\dot{\varepsilon})$, $X_T(\dot{\varepsilon})$, $Y_T(\dot{\varepsilon})$, $S(\dot{\varepsilon})$

Elastic moduli and strength values are interpolated from strain-rate-dependent tables. The constitutive update uses the orthotropic framework with rate-adjusted properties.

Application: Composites under impact and crash loading with rate-sensitive matrix behavior.

Material Library

Predefined material presets are available in `physics/material_library.hpp`:

Preset	E (GPa)	ν	ρ (kg/m^3)	σ_y (MPa)
Mild Steel	210	0.30	7,850	250
Aluminum 6061-T6	69	0.33	2,700	276
Titanium Ti-6Al-4V	114	0.34	4,430	880
Rubber (Neo-Hookean)	~1.5	0.4999	1,100	N/A
Concrete	30	0.20	2,400	N/A
EPS Foam	0.005	0.10	30	0.1

4.4.2 Failure Models

Eighteen failure models are provided in the `physics/failure/` directory. Each model implements the `FailureModel` interface and operates on the `FailureState` structure.

Failure Data Structures

```
struct FailureState {
    Real damage;            // 0 = intact, 1 = fully failed
    bool failed;
    int failure_mode;       // Model-specific failure mode index
    Real history[16];       // Model-specific history variables
};

class FailureModel {
public:
    virtual bool check_failure(const MaterialState& state,
                               FailureState& fstate) = 0;
    virtual void update_damage(const MaterialState& state,
                               FailureState& fstate, Real dt) = 0;
};
```

Failure Model Reference

Hashin Failure Criteria

Application: Unidirectional fiber-reinforced composites.

The Hashin model evaluates four independent failure modes:

Mode 1 — Fiber tension ($\sigma_{11} > 0$):

$$\left(\frac{\sigma_{11}}{X_T}\right)^2 + \frac{\sigma_{12}^2 + \sigma_{13}^2}{S_{12}^2} \geq 1$$

Mode 2 — Fiber compression ($\sigma_{11} < 0$):

$$\left(\frac{\sigma_{11}}{X_C}\right)^2 \geq 1$$

Mode 3 — Matrix tension ($\sigma_{22} + \sigma_{33} > 0$):

$$\frac{(\sigma_{22} + \sigma_{33})^2}{Y_T^2} + \frac{\sigma_{23}^2 - \sigma_{22}\sigma_{33}}{S_{23}^2} + \frac{\sigma_{12}^2 + \sigma_{13}^2}{S_{12}^2} \geq 1$$

Mode 4 — Matrix compression ($\sigma_{22} + \sigma_{33} < 0$):

$$\left[\left(\frac{Y_C}{2S_{23}}\right)^2 - 1\right]\frac{\sigma_{22} + \sigma_{33}}{Y_C} + \frac{(\sigma_{22} + \sigma_{33})^2}{4S_{23}^2} + \frac{\sigma_{12}^2 + \sigma_{13}^2}{S_{12}^2} \geq 1$$

Tsai-Wu Failure Criterion

Application: General composite laminates.

The Tsai-Wu criterion is a single quadratic interaction formula:

$$F_1\sigma_1 + F_2\sigma_2 + F_{11}\sigma_1^2 + F_{22}\sigma_2^2 + F_{66}\sigma_{12}^2 + 2F_{12}\sigma_1\sigma_2 \geq 1$$

where the strength parameters are:

$$F_1 = \frac{1}{X_T} - \frac{1}{X_C}, \quad F_2 = \frac{1}{Y_T} - \frac{1}{Y_C}, \quad F_{11} = \frac{1}{X_T X_C}, \quad F_{22} = \frac{1}{Y_T Y_C}, \quad F_{66} = \frac{1}{S^2}$$

The interaction term F_{12} is typically taken as $F_{12} = -\frac{1}{2}\sqrt{F_{11}F_{22}}$.

Chang-Chang Failure Criteria

Application: Laminated composites with in-situ strength effects.

The Chang-Chang model is a modified Hashin formulation that accounts for in-situ ply strengths, which differ from unidirectional test coupon strengths due to constraint effects.

GTN (Gurson-Tvergaard-Needleman)

Application: Ductile metal fracture.

Yield function:

$$\Phi = \left(\frac{q}{\sigma_y}\right)^2 + 2q_1 f^* \cosh\left(\frac{3q_2\, p}{2\sigma_y}\right) - \left(1 + (q_1 f^*)^2\right) = 0$$

where q is the von Mises stress, p is the hydrostatic pressure, σ_y is the matrix yield stress, and f^* is the effective void volume fraction (modified from the actual porosity f to model void coalescence).

Void evolution: $\dot{f} = \dot{f}_{\text{growth}} + \dot{f}_{\text{nucleation}}$

GISSMO (Generalized Incremental Stress-State Dependent)

Application: General-purpose ductile damage model.

Damage evolution:

$$\frac{\mathrm{d}D}{\mathrm{d}t} = \frac{n}{\varepsilon_f(\eta, \dot{\varepsilon})}\, D^{(n-1)/n}\, \dot{\varepsilon}^p$$

where n is the damage exponent and ε_f is the failure strain, which depends on the stress triaxiality η and strain rate $\dot{\varepsilon}$.

Tabulated Failure

Application: Materials characterized by experimental failure data.

The failure strain is specified as a tabulated function of stress triaxiality and strain rate: $\varepsilon_f = \varepsilon_f(\eta, \dot{\varepsilon})$. Bilinear interpolation is used between data points.

Johnson-Cook Failure

Parameters: D_1 through D_5, $\dot{\varepsilon}_0$, T_{melt}, T_{room}

Failure strain:

$$\varepsilon_f = [D_1 + D_2 \exp(D_3 \eta)] \left[1 + D_4 \ln \frac{\dot{\varepsilon}}{\dot{\varepsilon}_0}\right] [1 + D_5 T^*]$$

where $\eta = p/\sigma_{\text{vm}}$ is the stress triaxiality. Damage accumulates as $D = \sum \Delta\varepsilon^p / \varepsilon_f$.

Application: Ballistic impact, penetration, metal fragmentation.

Cockcroft-Latham Failure

Parameters: W_c (critical plastic work)

Failure criterion:

$$W = \int_0^{\bar{\varepsilon}^p} \max(\sigma_1, 0)\, d\bar{\varepsilon}^p \geq W_c$$

where σ_1 is the maximum principal stress. Failure occurs when the accumulated plastic work under tension reaches the critical value.

Application: Metal cutting, machining, ductile fracture under tension.

Lemaitre CDM (Continuum Damage Mechanics)

Parameters: S_0 (damage strength), s (damage exponent), D_c (critical damage), ε_D (damage threshold strain)

Damage evolution:

$$\dot{D} = \left(\frac{Y}{S_0}\right)^s \dot{\bar{\varepsilon}}^p, \qquad Y = \frac{\sigma_{\text{eq}}^2}{2E(1-D)^2} R_\nu$$

where $R_\nu = \frac{2}{3}(1+\nu) + 3(1-2\nu)\eta^2$ is the triaxiality function.

Application: Low-cycle fatigue, coupled damage-plasticity analyses.

Puck Failure Criteria

Parameters: $R_{\parallel}^{(+)}$, $R_{\parallel}^{(-)}$ (fiber strengths), $R_{\perp}^{(+)}$, $R_{\perp}^{(-)}$ (matrix strengths), $R_{\perp\parallel}$, $p_{\perp\parallel}^{(+)}$, $p_{\perp\parallel}^{(-)}$ (inclination parameters)

The Puck model distinguishes fiber failure (direct stress criterion) from inter-fiber failure (action-plane based) with separate modes for matrix tension, compression, and shear.

Application: Laminated composites with physically-based matrix failure characterization.

FLD (Forming Limit Diagram) Failure

Parameters: Forming limit curve as pairs $(\varepsilon_2, \varepsilon_1^{\text{limit}})$

Failure is detected when the in-plane principal strains $(\varepsilon_2, \varepsilon_1)$ cross the user-supplied forming limit curve. Linear interpolation is used between data points.

Application: Sheet metal stamping, formability assessment.

Wilkins Failure

Parameters: D_c (critical damage), a, β (pressure weight), w (deviatoric weight)

Damage accumulation:

$$D = \int \left(\frac{1}{1 - a\,P^*} \right)^{\beta} \left(\frac{2 - A_d}{A_d} \right)^{w} d\bar{\varepsilon}^p$$

where P^* is the normalized pressure and A_d is the asymmetry of the deviatoric stress tensor.

Application: Metal fragmentation, expanding ring/cylinder problems.

Tuler-Butcher Failure

Parameters: σ_c (threshold stress), K (spall integral limit), λ (exponent)

Spall criterion:

$$\int_0^t (\sigma_1 - \sigma_c)^{\lambda}\, dt \geq K$$

Failure occurs when the time-integrated excess tensile stress exceeds the material spall constant K.

Application: Spallation under explosive loading, plate impact.

Maximum Stress Failure

Parameters: X_T, X_C, Y_T, Y_C, Z_T, Z_C, S_{12}, S_{23}, S_{13} (component stress limits)

Failure occurs when any stress component exceeds its corresponding allowable: $\sigma_{11} > X_T$ or $|\sigma_{11}| > X_C$, etc. Each mode is tracked independently.

Application: Brittle materials, composites with non-interacting failure modes.

Maximum Strain Failure

Parameters: ε_1^T, ε_1^C, ε_2^T, ε_2^C, ε_3^T, ε_3^C, γ_{12}^f, γ_{23}^f, γ_{13}^f

Failure occurs when any strain component exceeds its corresponding allowable limit. Each mode is tracked independently, analogous to the maximum stress criterion but evaluated in strain space.

Application: Composites and ceramics where strain limits are better characterized than stress limits.

Energy-Based Failure

Parameters: W_c (critical strain energy density)

Failure criterion:

$$W = \frac{1}{2}\sigma_{ij}\varepsilon_{ij} \geq W_c$$

Failure occurs when the total strain energy density at an integration point reaches the critical threshold.

Application: General-purpose failure criterion for materials with known energy absorption capacity.

Wierzbicki Failure (Modified Mohr-Coulomb)

Parameters: c_1, c_2, c_θ^s, c_θ^{ax} (calibration constants)

Failure strain:

$$\varepsilon_f = \left(\frac{c_\theta^{ax}}{c_\theta}\right)\left[c_1 e^{-c_2\eta}\right]$$

where c_θ depends on the normalized Lode angle parameter $\bar{\theta}$, capturing the effect of stress state beyond triaxiality alone.

Application: Sheet metal fracture under complex stress states (bending, shear, biaxial).

Fabric Failure

Parameters: ε_1^f (warp failure strain), ε_2^f (weft failure strain), γ_{12}^f (shear failure strain)

Biaxial textile failure criterion with independent warp, weft, and shear failure modes. Upon failure in one direction, the corresponding stress component is zeroed while the remaining directions continue to carry load.

Application: Airbag tear, parachute failure, woven composite damage.

4.4.3 Equations of State

Thirteen equation-of-state (EOS) models are provided in `physics/eos.hpp`. These are used in conjunction with material models for problems involving high pressures, shock waves, or compressible media.

EOS Data Structure

```
struct EOSState {
    Real pressure;
    Real density, density_ref;
    Real internal_energy;
    Real sound_speed;
    Real temperature;
    Real compression;        //  = /0  1
};

class EquationOfState {
public:
    virtual void compute_pressure(EOSState& state) = 0;
    virtual Real sound_speed(const EOSState& state) = 0;
};
```

EOS Model Reference

Ideal Gas

$$P = (\gamma - 1)\,\rho\, e$$

where γ is the ratio of specific heats and e is the specific internal energy.

Application: Low-pressure gas dynamics.

Mie-Grüneisen

$$P = \frac{\rho_0 c_0^2 \mu \left[1 + \left(1 - \frac{\Gamma_0}{2}\right)\mu\right]}{\left[1 - (S_1 - 1)\mu - S_2 \frac{\mu^2}{\mu+1} - S_3 \frac{\mu^3}{(\mu+1)^2}\right]^2} + \Gamma_0 \rho e$$

where $\mu = \rho/\rho_0 - 1$, c_0 is the bulk sound speed, S_1, S_2, S_3 are the shock velocity–particle velocity coefficients, and Γ_0 is the Grüneisen parameter.

Application: Shock-loaded metals (copper, aluminum, steel).

JWL (Jones-Wilkins-Lee)

$$P = A\, e^{-R_1 V/V_0} + B\, e^{-R_2 V/V_0} + \frac{\omega\, e}{V}$$

where V/V_0 is the relative volume, and A, B, R_1, R_2, ω are material parameters.

Application: Detonation products from high explosives.

Polynomial

$$P = C_0 + C_1\mu + C_2\mu^2 + C_3\mu^3 + (C_4 + C_5\mu)\, e$$

Application: General-purpose EOS for a wide range of materials.

Tabulated

$$P = P(\rho, e) \quad \text{from lookup table}$$

Bilinear interpolation on a regular grid of density and internal energy values.

Application: Materials with experimentally measured EOS data.

Murnaghan

Parameters: K_0 (bulk modulus), K_0' (pressure derivative), ρ_0

$$P = \frac{K_0}{K_0'}\left[\left(\frac{\rho}{\rho_0}\right)^{K_0'} - 1\right]$$

Application: Third-order elastic description of solids at moderate pressures.

Noble-Abel

Parameters: γ (ratio of specific heats), b (covolume)

$$P = \frac{(\gamma - 1)\rho\, e}{1 - b\,\rho}$$

The covolume b accounts for the finite molecular volume, modifying the ideal gas law at high densities.

Application: Propellant gases, combustion products at high density.

Stiff Gas (Tait)

Parameters: B (pressure constant), γ (Tait exponent), ρ_0

$$P = B\left[\left(\frac{\rho}{\rho_0}\right)^{\gamma} - 1\right]$$

Application: Water and other weakly compressible liquids, underwater explosions.

Tillotson

Parameters: a, b, A, B, E_0, α, β, E_{iv}, E_{cv}

The Tillotson EOS uses separate formulations for compressed, cold expanded, and hot expanded states, with a smooth interpolation region between incipient and complete vaporization energies (E_{iv} and E_{cv}).

Application: Hypervelocity impact, planetary collision, meteorite impact simulations.

Sesame

Parameters: 2D lookup table $P(\rho, T)$, $e(\rho, T)$

Pressure and energy are obtained by bilinear interpolation from a 2D thermodynamic table (typically from the SESAME database). Temperature and density serve as the independent variables.

Application: Multi-phase materials, high-energy-density physics, weapon effects.

PowderBurn

Parameters: ρ_s (solid density), A_c (burn rate coefficient), n (pressure exponent), EOS parameters for gas and solid phases

$$\dot{F} = A_c P^n (1 - F)^{2/3}$$

where F is the burned mass fraction. The model couples a surface-regression burn rate with separate EOS for the solid propellant and product gas phases.

Application: Propellant combustion, gun interior ballistics, reactive materials.

Compaction

Parameters: Loading curve $P(\rho)$, unloading bulk modulus K_u

The compaction EOS follows a nonlinear loading path defined by a tabulated pressure-density curve. Unloading is elastic with a user-specified bulk modulus, producing permanent densification.

Application: Porous metals, powder compacts, geological compaction.

Osborne

Parameters: A_1 through A_6 (polynomial coefficients)

$$P = \frac{A_1\mu + A_2\mu^2 + A_3\mu^3 + (A_4 + A_5\mu + A_6\mu^2)e}{e + 1}$$

where $\mu = \rho/\rho_0 - 1$. This extended polynomial form provides stronger coupling between compression and internal energy than the standard polynomial EOS.

Application: Metals and geomaterials requiring pressure-energy coupling.

4.4.4 Composite Analysis

The composite analysis module (`physics/composite/`) implements Classical Lamination Theory (CLT) with extensions for thermal residual stress, interlaminar shear stress, and progressive failure analysis.

Ply Definition

```
struct Ply {
    Real thickness;
    Real angle;                // Fiber orientation (degrees)
    Real E1, E2, G12, nu12;   // Lamina elastic properties
    Real Xt, Xc, Yt, Yc, S;   // Strength properties
    Real alpha1, alpha2;       // Coefficients of thermal expansion
};
```

ABD Matrix Computation

The `CompositeLaminate` class computes the ABD stiffness matrix from the laminate definition:

$$\begin{bmatrix} \mathbf{N} \\ \mathbf{M} \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B} & \mathbf{D} \end{bmatrix} \begin{bmatrix} \boldsymbol{\varepsilon}^0 \\ \boldsymbol{\kappa} \end{bmatrix}$$

where:

$$A_{ij} = \sum_{k=1}^{N} \bar{Q}_{ij}^{(k)} (z_k - z_{k-1})$$

$$B_{ij} = \frac{1}{2} \sum_{k=1}^{N} \bar{Q}_{ij}^{(k)} (z_k^2 - z_{k-1}^2)$$

$$D_{ij} = \frac{1}{3} \sum_{k=1}^{N} \bar{Q}_{ij}^{(k)} (z_k^3 - z_{k-1}^3)$$

Here $\bar{Q}_{ij}^{(k)}$ is the transformed reduced stiffness matrix for ply k, and z_k is the distance from the laminate midplane to the top of ply k.

Thermal Residual Stress

The `CompositeThermal` class computes thermal resultants and ply-level stresses due to a temperature change ΔT:

$$\mathbf{N}^T = \sum_{k=1}^{N} \bar{\mathbf{Q}}^{(k)} \boldsymbol{\alpha}^{(k)} \Delta T \, (z_k - z_{k-1})$$

$$\mathbf{M}^T = \frac{1}{2} \sum_{k=1}^{N} \bar{\mathbf{Q}}^{(k)} \boldsymbol{\alpha}^{(k)} \Delta T \, (z_k^2 - z_{k-1}^2)$$

Interlaminar Shear Stress

The `CompositeInterlaminar` class computes through-thickness shear stresses using equilibrium equations, which are critical for delamination assessment.

Progressive Failure Analysis

The `CompositeProgressiveFailure` class implements:

1. **First-Ply Failure (FPF)** — Detection of the first ply to fail under applied loading using Hashin, Tsai-Wu, or maximum stress criteria.
2. **Ply degradation** — After failure, the stiffnesses of the failed ply are reduced:
   - **Selective degradation:** Only the moduli associated with the failure mode are set to near-zero (e.g., fiber failure zeros E_1; matrix failure zeros E_2 and G_{12}).
   - **Discount method:** All stiffnesses are set to near-zero.
3. **Strength envelope generation** — A polar plot of failure load as a function of loading angle, computed by evaluating the failure criterion at multiple load orientations.

4.5 Part V — Solvers

This part describes the explicit dynamics solver, the implicit solver framework, and the nonlinear solution methods available in NexusSim.

4.5.1 Explicit Dynamics

The explicit dynamics solver (`fem/fem_solver.hpp`, `physics/time_integration.hpp`, `physics/adaptive_timestep.hpp`) implements the central-difference time integration scheme, which is conditionally stable and well-suited for short-duration, high-deformation events such as impact, blast, and crash simulations.

Central Difference Algorithm

Given the mass matrix $\mathbf{M}$, displacements $\mathbf{u}_n$, velocities $\mathbf{v}_n$, accelerations $\mathbf{a}_n$, and time step Δt, the algorithm proceeds as follows:

1. Compute accelerations: $$ \mathbf{a}*n* = *mathbf{M}^{-1}* *left(\mathbf{F}*{\text{ext}} - \mathbf{F}_{\text{int}}
- \mathbf{F}_{\text{contact}} - \mathbf{C}\mathbf{v}_n\right) $$
2. Update velocities (half-step): $\mathbf{v}_{n+1/2} = \mathbf{v}_{n-1/2} + \Delta t\, \mathbf{a}_n$
3. Update displacements: $\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\, \mathbf{v}_{n+1/2}$
4. Compute internal forces from the updated configuration: $\mathbf{F}_{\text{int}}(\mathbf{u}_{n+1})$.
5. Apply contact forces, erosion criteria, and boundary conditions.

The use of a lumped (diagonal) mass matrix $\mathbf{M}$ makes the inversion trivial and avoids the need to solve a linear system at each time step.

Adaptive Time Step

The stable time step is determined by the CFL (Courant–Friedrichs–Lewy) condition:

$$\Delta t = f_s \cdot \min_e \frac{L_e}{c_e}$$

where L_e is the characteristic length of element e, $c_e = \sqrt{E/\rho}$ is the wave speed, and $f_s = 0.9$ is the safety factor.

```
struct AdaptiveTimestep {
    Real compute_stable_dt(const Mesh& mesh, const State& state);
};
```

Subcycling

The `SubcyclingController` class allows different element groups to be integrated at different time steps. Small or stiff elements that impose a restrictive time step are subcycled at a smaller Δt, while the remainder of the mesh uses the global Δt.

```
class SubcyclingController {
    void add_group(const std::string& name, Real dt_factor);
    int subcycle_ratio(const std::string& group);
};
```

Energy Monitor

The `EnergyMonitor` class tracks energy balance throughout the simulation:

```
class EnergyMonitor {
    Real kinetic_energy();
    Real internal_energy();
    Real contact_energy();
    Real hourglass_energy();
    Real total_energy();
    Real energy_error();    // (E_total  E_initial) / E_initial
};
```

Energy balance is a primary indicator of simulation quality. A well-conditioned explicit simulation should maintain energy error below 5–10%.

Element Erosion

The `ElementErosion` class (`physics/element_erosion.hpp`) provides over 15 failure criteria for removing severely distorted elements from the simulation:

```
class ElementErosion {
    void check_erosion(State& state, Real dt);
    void erode_element(Index elem_id);
};
```

Available criteria: Maximum principal strain, von Mises stress, hydrostatic pressure, time step ratio, volume change, NaN detection, and user-defined criteria.

When an element is eroded, its mass is redistributed to its nodes and it ceases to contribute internal forces.

Wave 9: Explicit Solver Enhancements

The explicit solver was completed in Wave 9 (`fem/explicit_dynamics.hpp`) with the following additions:

Bulk Viscosity — Linear and quadratic artificial viscosity for shock capturing:

$$q = C_1 \rho c L_e |\dot{\varepsilon}_v| + C_2 \rho L_e^2 \dot{\varepsilon}_v^2$$

where C_1 and C_2 are the linear and quadratic viscosity coefficients.

Hourglass Control — Flanagan-Belytschko viscous and perturbation-based stiffness hourglass stabilization for reduced-integration elements.

Energy Monitoring — Per-step tracking of kinetic, internal, contact, hourglass, and external work energies with energy balance error checking.

Element Erosion — Stress/strain-based element deletion with mass conservation and neighbor contact update.

4.5.2 Implicit Solvers

The implicit solver framework (`solver/` directory) provides the infrastructure for static and dynamic analyses that require the solution of large sparse linear systems.

Sparse Matrix (CSR Format)

The `SparseMatrixCSR` class (`solver/sparse_matrix.hpp`) stores the global stiffness matrix in Compressed Sparse Row (CSR) format:

```
class SparseMatrixCSR {
public:
    SparseMatrixCSR(Index rows, Index cols);
    void build_from_coo(rows, cols, values);
    void add_value(Index row, Index col, Real value);
    void multiply(const std::vector<Real>& x, std::vector<Real>& y) const;
    Real diagonal(Index i) const;
    void scale(Real alpha);
    const std::vector<Index>& row_ptr() const;
    const std::vector<Index>& col_indices() const;
    std::vector<Real>& values();
};
```

A GPU-compatible sparse matrix class is also available (`solver/gpu_sparse_matrix.hpp`) for systems compiled with Kokkos GPU backends.

Linear Solvers

Preconditioned Conjugate Gradient

The `ConjugateGradientSolver` class implements the preconditioned conjugate gradient (PCG) method for symmetric positive-definite systems:

```
class ConjugateGradientSolver {
    struct Config {
        int max_iterations = 1000;
        Real tolerance = 1.0e-10;
        bool verbose = false;
    };

    SolverResult solve(const SparseMatrixCSR& A,
                       const std::vector<Real>& b,
                       std::vector<Real>& x,
                       const Preconditioner* precond = nullptr);
};
```

Available preconditioners:

Preconditioner	Description
`JacobiPreconditioner`	Diagonal scaling: $M = \text{diag}(A)$
`SSORPreconditioner`	Symmetric SOR with relaxation parameter ω

Direct Solver

The `DirectSolver` class performs LU factorization for small-to-medium systems:

```
class DirectSolver {
    SolverResult solve(const SparseMatrixCSR& A,
                       const std::vector<Real>& b,
                       std::vector<Real>& x);
};
```

Newton–Raphson Solver

The `NewtonRaphsonSolver` class (`solver/implicit_solver.hpp`) solves nonlinear systems using Newton's method with optional line search:

```
class NewtonRaphsonSolver {
    using InternalForceFunction =
        std::function<std::vector<Real>(const std::vector<Real>& u)>;
    using TangentFunction =
        std::function<SparseMatrixCSR(const std::vector<Real>& u)>;

    struct Config {
        int max_iterations = 20;
        Real tolerance = 1.0e-8;
        Real line_search_alpha = 1.0;
        bool use_line_search = true;
    };

    NewtonResult solve(const std::vector<Real>& F_ext,
                       std::vector<Real>& u,
                       InternalForceFunction F_int,
                       TangentFunction K_tangent);
};
```

The internal force and tangent stiffness are provided as callbacks, decoupling the solver from the element-level computations.

FEM Static Solver

The `FEMStaticSolver` class (`solver/fem_static_solver.hpp`) provides a high-level interface for linear and nonlinear static analyses:

```
class FEMStaticSolver {
public:
    FEMStaticSolver(Mesh& mesh);

    // Material assignment
    void set_material(Real E, Real nu);
    void set_shell_thickness(Index elem_id, Real thickness);

    // Boundary conditions
    void fix_node(Index node_id, int dof, Real value = 0.0);
    void fix_node_all(Index node_id);
    void add_force(Index node_id, int dof, Real value);
    void add_moment(Index node_id, int dof, Real value);

    // Solution
    void solve_linear();
    ArcLengthResult solve_arc_length(ArcLengthConfig config);

    // Post-processing
    std::vector<Real> get_node_displacement(Index node_id, int dpn = 3);
    Real max_displacement(int dpn = 3);
    int dof_per_node();
};
```

Element dispatch: The solver automatically detects element types in the mesh and dispatches to the corresponding stiffness matrix routine. Supported element types are Hex8, Hex20, Tet4, Tet10, and Shell4.

Shell4 6-DOF auto-detection: When shell elements are present in the mesh, the solver automatically switches to 6 DOFs per node. Solid-only nodes in mixed meshes receive a rotational penalty stiffness to prevent singularity.

FEM Implicit Dynamic Solver

The `FEMImplicitDynamicSolver` class solves the transient equation of motion using the Newmark-β time integration scheme:

$$\mathbf{M}\ddot{\mathbf{u}}_{n+1} + \mathbf{C}\dot{\mathbf{u}}_{n+1} + \mathbf{K}\mathbf{u}_{n+1} = \mathbf{F}_{n+1}$$

Newmark-β update equations:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t\, \dot{\mathbf{u}}_n + \Delta t^2 \left[(0.5 - \beta)\ddot{\mathbf{u}}_n + \beta\, \ddot{\mathbf{u}}_{n+1}\right]$$

$$\dot{\mathbf{u}}_{n+1} = \dot{\mathbf{u}}_n + \Delta t \left[(1 - \gamma)\ddot{\mathbf{u}}_n + \gamma\, \ddot{\mathbf{u}}_{n+1}\right]$$

Standard parameters: $\beta = 0.25$ (average acceleration, unconditionally stable), $\gamma = 0.5$ (no numerical damping).

Effective stiffness: $\mathbf{K}_{\text{eff}} = \mathbf{K} + a_0\mathbf{M} + a_1\mathbf{C}$

Robustness Guards

The implicit solver includes comprehensive diagnostic checks:

- **NaN/Inf detection** — Applied to the right-hand-side vector, residual, $\mathbf{p}^T\mathbf{A}\mathbf{p}$ dot product, and solution vector.
- **Zero diagonal detection** — Identifies unconstrained degrees of freedom.
- **Degenerate element skip** — Elements with negative or zero Jacobian determinant are skipped during assembly.
- **Condition number estimate** — A warning is issued for ill-conditioned systems.
- **Solver diagnostics** — Detailed diagnostic messages are generated upon convergence failure.

4.5.3 Nonlinear Solution Methods

Arc-Length Method (Crisfield's Cylindrical)

The arc-length solver (`solver/arc_length_solver.hpp`) traces nonlinear equilibrium paths, including snap-through and snap-back behavior, by treating the load factor λ as an additional unknown.

Constraint equation (cylindrical, $\psi = 0$):

$$\|\Delta\mathbf{u}\|^2 + \psi^2(\Delta\lambda)^2\|\mathbf{F}_{\text{ref}}\|^2 = \Delta\ell^2$$

Configuration:

```
struct ArcLengthConfig {
    Real arc_length = 1.0;
    Real psi = 0.0;                // 0 = cylindrical constraint
    int max_steps = 100;
    Real tolerance = 1.0e-6;
    int max_corrections = 15;
    Real lambda_max = 2.0;
    int desired_iterations = 5;    // For adaptive step sizing
    Real min_arc_length, max_arc_length;
    bool verbose = false;
};
```

Algorithm (per step):

1. **Predictor:** Solve $\mathbf{K}_t\,\delta\mathbf{u}_t = \mathbf{F}_{\text{ref}}$; compute $\Delta\lambda = \pm\Delta\ell\,/\,\|\delta\mathbf{u}_t\|$.
2. **Corrector (bordering):** Solve $\mathbf{K}_t\,\delta\mathbf{u}_R = -\mathbf{R}$ and $\mathbf{K}_t\,\delta\mathbf{u}_F = \mathbf{F}_{\text{ref}}$; determine $\mathrm{d}\lambda$ from the quadratic constraint equation.
3. **Convergence check:** $\|\mathbf{R}\|\,/\,\|\mathbf{F}_{\text{ref}}\| <$ tolerance.
4. **Adaptive step:** $\Delta\ell_{\text{new}} = \Delta\ell\sqrt{n_{\text{desired}}\,/\,n_{\text{actual}}}$, clamped to $[\Delta\ell_{\min}, \Delta\ell_{\max}]$.

Quadratic for $\mathrm{d}\lambda$:

$$a\,\mathrm{d}\lambda^2 + b\,\mathrm{d}\lambda + c = 0$$

where:

$$a = \delta\mathbf{u}_F \cdot \delta\mathbf{u}_F + \psi^2 \|\mathbf{F}_{\text{ref}}\|^2$$

$$b = 2(\Delta\mathbf{u} + \delta\mathbf{u}_R) \cdot \delta\mathbf{u}_F + 2\psi^2 \Delta\lambda \|\mathbf{F}_{\text{ref}}\|^2$$

$$c = (\Delta\mathbf{u} + \delta\mathbf{u}_R) \cdot (\Delta\mathbf{u} + \delta\mathbf{u}_R) + \psi^2 \Delta\lambda^2 \|\mathbf{F}_{\text{ref}}\|^2 - \Delta\ell^2$$

Result structure:

```
struct ArcLengthResult {
    bool converged;
    int total_steps, total_iterations;
    Real final_load_factor;
    std::vector<PathPoint> path;   // (lambda, displacement, iterations) per step
};
```

PETSc Integration

When PETSc is available (`NEXUSSIM_HAVE_PETSC`), the `PETScLinearSolver` class (`solver/petsc_solver.hpp`) provides access to PETSc's scalable linear solvers:

```
class PETScLinearSolver : public LinearSolver {
    void set_ksp_type(PETScKSPType type);
    void set_pc_type(PETScPCType type);
    SolverResult solve(const SparseMatrixCSR& A,
                       const std::vector<Real>& b,
                       std::vector<Real>& x);
};
```

Supported Krylov subspace (KSP) types: CG, GMRES, BiCGSTAB, PREONLY.

Supported preconditioner (PC) types: None, Jacobi, ILU, ICC, AMG, LU, Cholesky.

The solver converts NexusSim's CSR format to PETSc `Mat` and `Vec` objects internally and calls `KSPSolve`. PETSc support is enabled via `cmake -DNEXUSSIM_ENABLE_PETSC=ON`.

4.6 Part VI — Contact and Constraints

This part describes the contact mechanics formulations, rigid body dynamics and kinematic constraints, and the load and boundary condition subsystems.

4.6.1 Contact Mechanics

The contact module (`fem/` directory) provides sixteen contact formulations. These are organized from the simplest penalty-based methods to the most sophisticated mortar and tied formulations.

Contact Types

Contact Type	Method	Governing Relation
Penalty	Node-to-surface	$F = k_n \cdot g$ (gap function)
Node-to-Surface	Projection	Node projected onto master surface
Surface-to-Surface	Segment-based	Full segment pair interaction
Hertzian	Analytical	$F = k\,\delta^{3/2}$ (sphere–sphere)
Mortar	Segment-to-segment	D/M integrals, augmented Lagrangian
Tied	Penalty/constraint	Permanent bonding with optional failure
Voxel Collision	Grid-based	Spatial hashing for broad-phase detection

Contact Interface

All contact types implement the following base interface:

```
class Contact {
public:
    void detect(const Mesh& mesh, const State& state);
    void enforce(State& state, Real dt);
    void set_penalty_stiffness(Real k);
    void set_friction(Real mu_s, Real mu_d);
};
```

Friction Model

The `FrictionModel` class (`fem/friction_model.hpp`) implements Coulomb friction with exponential decay from static to dynamic coefficients:

$$F_t = \min(\mu\, F_n,\ \|F_{\text{trial}}\|)$$

where:

$$\mu = \mu_d + (\mu_s - \mu_d)\, e^{-\alpha \|\dot{u}_t\|}$$

```
class FrictionModel {
    Real mu_static;
    Real mu_dynamic;
    Real decay_coefficient;
};
```

Hertzian Contact

The `HertzianContact` class (`fem/hertzian_contact.hpp`) implements analytical Hertz contact theory for sphere–sphere or sphere–plane interactions:

$$F = \frac{4}{3} E^* \sqrt{R^*}\, \delta^{3/2}$$

where the effective modulus and radius are:

$$\frac{1}{E^*} = \frac{1-\nu_1^2}{E_1} + \frac{1-\nu_2^2}{E_2}, \qquad \frac{1}{R^*} = \frac{1}{R_1} + \frac{1}{R_2}$$

Contact damping follows the Hunt–Crossley model:

$$F_d = \alpha\, \delta^{3/2}\, \dot{\delta}$$

where α is derived from the coefficient of restitution.

Mortar Contact

The `MortarContact` class (`fem/mortar_contact.hpp`) implements a segment-to-segment mortar formulation with augmented Lagrangian enforcement:

1. **Segment clipping** — Polygon clipping determines the overlapping region between slave and master segments.
2. **D/M integrals** — The mortar integrals D_{ij} and M_{ij} are evaluated over the clipped region using Gauss quadrature.
3. **Contact stiffness** — The contact contribution to the global stiffness matrix is assembled from the mortar integrals.
4. **Augmented Lagrangian update** — The Lagrange multipliers are updated iteratively: $\lambda_{n+1} = \lambda_n + \epsilon_n\, g_n$

Tied Contact

The `TiedContact` class (`fem/tied_contact.hpp`) enforces permanent bonding between slave and master surfaces using penalty stiffness:

```
class TiedContact {
    void tie_surfaces(const std::vector<Index>& slave,
                      const std::vector<Index>& master);
    void set_failure_stress(Real sigma_n, Real sigma_s);
    void set_failure_strain(Real eps_n, Real eps_s);
    void check_failure(const State& state);
};
```

Tied-with-failure: When failure stresses or strains are specified, the tied constraint monitors the interface forces. When the normal or shear stress exceeds the specified threshold, the bond is broken and the nodes are released.

Voxel Collision Detection

The `VoxelCollision` class (`fem/voxel_collision.hpp`) provides a grid-based broad-phase collision detection algorithm using spatial hashing. It is designed for efficient detection of potential contact pairs before the narrow-phase contact algorithms are applied.

Wave 12: Extended Contact Formulations

The following additional contact capabilities are available in `fem/contact_wave12.hpp` (namespace `nxs::fem`):

Contact Stiffness Scaler — Adaptive penalty stiffness that scales automatically based on the local element size, material modulus, and time step. Prevents both excessive penetration (too soft) and numerical instability (too stiff).

Velocity-Dependent Friction — Extends the Coulomb model with an explicit static-to-kinetic transition governed by relative sliding velocity: $\mu(v) = \mu_d + (\mu_s - \mu_d)\, e^{-\alpha v}$. The decay coefficient α is user-configurable.

Shell Thickness Contact — Accounts for shell element thickness offsets when computing contact gaps. The mid-surface position is offset by $t/2$ along the element normal so that contact occurs at the physical outer surface rather than at the reference surface.

Edge-to-Edge Contact — Detects contact between beam or shell edge pairs using closest-point projection between line segments. Computes gap, normal direction, and penalty force for edge–edge interactions that node-to-surface methods miss.

Segment-Based Contact — Face-to-face penalty formulation using a bucket-sort spatial search for broad-phase pair detection. Each segment pair is checked for overlap; penalty forces are distributed to all contributing nodes.

Self-Contact — Single-surface contact detection where one surface can contact itself. Uses spatial hashing to identify candidate segment pairs and excludes adjacent elements to avoid false positives.

Symmetric Contact — Bidirectional contact where both surfaces are simultaneously treated as master and slave. Averaging of the two pass results eliminates the master/slave bias present in standard one-pass algorithms.

Rigid-Deformable Contact — Specialized interface between rigid bodies and deformable elements. Rigid surface nodes are treated as prescribed-motion master nodes, and contact forces are accumulated into the rigid body force/moment resultants.

Multi-Surface Contact Manager — Automatic surface detection and dispatch layer that assigns contact pairs to the appropriate algorithm based on surface type (shell, solid, rigid, beam). Manages a global contact table and invokes detection/enforcement for all active pairs each time step.

4.6.2 Rigid Bodies and Constraints

Rigid Bodies

The `RigidBody` class (`fem/rigid_body.hpp`) models rigid body dynamics using the Newton–Euler equations:

$$M\,\mathbf{a}_{\text{cm}} = \mathbf{F}, \qquad \mathbf{I}\,\boldsymbol{\alpha} = \boldsymbol{\tau}$$

where M is the total mass, $\mathbf{a}_{\text{cm}}$ is the center-of-mass acceleration, $\mathbf{I}$ is the inertia tensor, and $\boldsymbol{\alpha}$ is the angular acceleration.

```
class RigidBody {
    void set_mass(Real m);
    void set_inertia(const Mat3& I);
    void set_center_of_mass(const Vec3r& cm);
    void compute_rigid_motion(Real dt);
    void update_node_positions();
    Vec3r angular_momentum() const;
    Vec3r angular_velocity() const;
};
```

Rigid Walls

The `RigidWall` class (`fem/rigid_wall.hpp`) provides four wall geometries for constraining node motion:

Wall Type	Geometry	Application
Planar	Infinite plane with normal	Simple barriers
Cylindrical	Cylinder with axis and radius	Pipes, containment
Spherical	Sphere with center and radius	Hemispherical dies
Moving	Prescribed-velocity wall	Crush simulations

```
class RigidWall {
    void set_wall_type(WallType type);
    void set_velocity(const Vec3r& v);
    Real compute_gap(const Vec3r& point) const;
    Vec3r compute_normal(const Vec3r& point) const;
    void enforce(State& state, Real dt);
};
```

Kinematic Constraints

The `Constraints` class (`fem/constraints.hpp`) implements multi-point constraints and joints:

Constraint	Description
RBE2	Rigid spider: slave nodes follow master rigidly
RBE3	Interpolation: master DOFs = weighted average of slaves
Revolute joint	One rotational DOF free
Spherical joint	Three rotational DOFs free
Cylindrical joint	One rotation + one translation free

```
class Constraints {
    void add_rbe2(Index master, const std::vector<Index>& slaves);
    void add_rbe3(Index master, const std::vector<Index>& slaves,
                  const std::vector<Real>& weights);
    void add_joint(JointType type, Index node_a, Index node_b,
                   const Vec3r& axis);
    void enforce(State& state, Real dt);
};
```

4.6.3 Loads and Boundary Conditions

Load Curves

The `LoadCurve` class (`fem/load_curve.hpp`) provides piecewise-linear time functions for scaling loads:

```
class LoadCurve {
    void add_point(Real time, Real value);
    Real evaluate(Real time) const;
    void set_extrapolation(Extrapolation mode);
};
```

Extrapolation modes: `Zero` (return 0 outside range), `Constant` (hold last value), `Linear` (continue slope).

The `LoadCurveManager` class provides a centralized registry:

```
class LoadCurveManager {
    Index add_curve(const LoadCurve& curve);
    Real evaluate(Index curve_id, Real time);
};
```

Load Types

Load Type	Description
Nodal force	Point force at a node (F_x, F_y, F_z)
Distributed load	Force per unit length or area
Body force	Acceleration field (e.g., gravity)
Pressure	Normal force per unit area on surfaces
Prescribed motion	Enforced displacement, velocity, or acceleration

```
class LoadManager {
    void add_nodal_force(Index node, int dof, Real value, Index curve_id = -1);
    void add_body_force(const Vec3r& acceleration);
    void add_pressure(const std::vector<Index>& face_nodes, Real pressure);
    void compute_external_forces(std::vector<Real>& F_ext, Real time);
};
```

Initial Conditions

The `InitialConditions` class (`fem/initial_conditions.hpp`) specifies the state at $t = 0$:

```
class InitialConditions {
    void set_velocity(Index node, const Vec3r& v);
    void set_velocity_field(const std::vector<Index>& nodes, const Vec3r& v);
    void set_temperature(Index node, Real T);
    void apply(State& state);
};
```

4.7 Part VII — Multi-Physics

This part describes the meshfree methods (SPH and Peridynamics), the multi-physics coupling framework, and the thermal analysis module.

4.7.1 Smoothed Particle Hydrodynamics

The SPH module (`sph/` directory) implements a weakly compressible SPH (WCSPH) formulation for fluid and free-surface flow problems.

SPH Kernels

The kernel function $W(r, h)$ is the fundamental building block of SPH interpolation. Three kernels are provided:

Kernel	Continuity	Support	Properties
Cubic B-spline	C^2	$2h$	Standard SPH kernel
Wendland C4	C^4	$2h$	Higher smoothness, less tensile instability
Quintic	C^4	$3h$	Very smooth, larger support

```
class SPHKernel {
    Real W(Real r, Real h) const;         // Kernel value
    Real dWdr(Real r, Real h) const;      // Kernel gradient
    Real support_radius(Real h) const;    // Compact support radius
};
```

SPH Solver

The `SPHSolver` class implements the WCSPH algorithm:

```
class SPHSolver {
    void compute_density();      // _i = _j m_j W(|x_i  x_j|, h)
    void compute_forces();       // Pressure + viscosity + body forces
    void time_integrate(Real dt);
};
```

Density summation:

$$\rho_i = \sum_j m_j \, W(|\mathbf{x}_i - \mathbf{x}_j|, h)$$

Momentum equation:

$$\frac{\mathrm{d}\mathbf{v}_i}{\mathrm{d}t} = -\sum_j m_j \left(\frac{P_i}{\rho_i^2} + \frac{P_j}{\rho_j^2} \right) \nabla W_{ij} + \mathbf{g}$$

Equation of state (Tait):

$$P = \frac{c^2 \rho_0}{\gamma} \left[\left(\frac{\rho}{\rho_0} \right)^\gamma - 1 \right] \approx c^2(\rho - \rho_0)$$

Neighbor Search

The `NeighborSearch` class (`sph/neighbor_search.hpp`) implements spatial hashing for efficient neighbor finding within the kernel support radius.

FEM-SPH Coupling

The `FEMSPHCoupling` class (`sph/fem_sph_coupling.hpp`) enables coupling between the FEM domain and the SPH domain at their interface:

```
class FEMSPHCoupling {
    void detect_interface(const Mesh& fem_mesh, const SPHParticles& sph);
    void transfer_pressure();      // SPH pressure → FEM surface loads
    void transfer_velocity();      // FEM velocity → SPH boundary particles
};
```

Coupling methods:

- **Penalty:** Spring forces at the interface proportional to penetration.
- **Pressure:** Direct pressure interpolation between the FEM surface and SPH particles.

4.7.2 Peridynamics

The peridynamics module (`peridynamics/` directory) provides a complete implementation of bond-based, ordinary state-based, and non-ordinary correspondence peridynamics with specialized constitutive models for metals, ceramics, and geomaterials.

Data Structures

Peridynamics uses Kokkos views for GPU-portable data:

```
using PDScalarView = Kokkos::View<Real*>;
using PDVectorView = Kokkos::View<Real*[3]>;
using PDIntView    = Kokkos::View<Index*>;
using PDBoolView   = Kokkos::View<bool*>;
```

PDParticles stores the particle state:

View	Description
`x()`	Reference positions
`u()`	Displacements
`v()`	Velocities
`f()`	Forces
`volume()`	Particle volumes
`density()`	Material densities
`active()`	Active/inactive flags

PDNeighborList stores the bond connectivity in CSR format:

View	Description
`neighbor_offset()`	CSR-style row offsets
`neighbor_list()`	Neighbor particle indices
`neighbor_count()`	Number of neighbors per particle
`bond_weight()`	Influence function weights
`bond_intact()`	Bond integrity flags
`bond_xi()`	Reference bond vectors $\boldsymbol{\xi} = \mathbf{x}_j - \mathbf{x}_i$
`bond_length()`	Reference bond lengths $

Bond-Based Peridynamics (PMB Model)

The Prototype Microelastic Brittle (PMB) model computes bond forces as:

$$\mathbf{f}(\boldsymbol{\xi}, \boldsymbol{\eta}) = c\, s\, w(|\boldsymbol{\xi}|)\, \mathbf{e}$$

where:

- $\boldsymbol{\xi} = \mathbf{x}_j - \mathbf{x}_i$ is the reference bond vector.
- $\boldsymbol{\eta} = \mathbf{u}_j - \mathbf{u}_i$ is the relative displacement.
- $s = (|\boldsymbol{\xi} + \boldsymbol{\eta}| - |\boldsymbol{\xi}|)/|\boldsymbol{\xi}|$ is the bond stretch.
- $\mathbf{e} = (\boldsymbol{\xi} + \boldsymbol{\eta})/|\boldsymbol{\xi} + \boldsymbol{\eta}|$ is the unit direction.
- $c = 18K/(\pi\delta^4)$ is the three-dimensional micro-modulus.

- w is the influence function.

Bond failure: A bond breaks irreversibly when $s > s_c$, where:

$$s_c = \sqrt{\frac{5G_c}{9K\delta}}$$

and G_c is the critical energy release rate.

Ordinary State-Based Peridynamics

The ordinary state-based formulation separates volumetric and deviatoric deformation:

$$\mathbf{f} = \frac{3K\theta}{m} w|\boldsymbol{\xi}|\,\mathbf{e} + \frac{15G}{m} w \left(|\boldsymbol{\eta} + \boldsymbol{\xi}| - |\boldsymbol{\xi}| \left(1 + \frac{\theta}{3}\right)\right) \mathbf{e}$$

where θ is the dilatation and m is the weighted volume.

Non-Ordinary Correspondence

The correspondence formulation computes the deformation gradient $\mathbf{F}$ from peridynamic kinematics and applies standard continuum constitutive models:

$$\mathbf{F} = \left[\sum_j (\boldsymbol{\eta}_j \otimes \boldsymbol{\xi}_j) V_j w_j \right] \mathbf{K}^{-1}$$

where $\mathbf{K} = \sum_j (\boldsymbol{\xi}_j \otimes \boldsymbol{\xi}_j) V_j w_j$ is the shape tensor.

Available constitutive models: Neo-Hookean, linear elastic, Johnson-Cook.

Hourglass control: Penalty-based stabilization suppresses zero-energy modes inherent in the correspondence formulation.

Enhanced Bond Models

Model	Description	History Variables
Energy-Based	Bond breaks when accumulated strain energy > $G_c	\boldsymbol{\xi}
Microplastic	Elasto-plastic bond with yield stretch	`history[0]` = plastic stretch
Viscoelastic	Standard linear solid with relaxation	`history[1]` = viscous variable
Short-Range Repulsion	Compressive contact force only	None

PD Material Models

Three specialized PD material models are available:

Johnson-Cook — For metals at high strain rates, with damage parameters D_1–D_5.

Drucker-Prager — For geomaterials (soil, rock), parameterized by friction angle ϕ, cohesion c, and dilation angle ψ.

Johnson-Holmquist 2 (JH2) — For ceramics and glass, with separate intact and fractured strength surfaces, progressive damage, and a polynomial EOS.

Material presets: Al7075-T6, Steel 4340, Ti-6Al-4V, Copper OFHC, LooseSand, DenseSand, Clay, Concrete, Granite, Alumina, SiC, B4C, Soda-Lime Glass.

4.7.3 Multi-Physics Coupling

The coupling framework (`coupling/` directory) enables data transfer between different physics modules. It is used for FEM–SPH and FEM–PD coupling.

Field Registry

The `FieldRegistry` singleton (`coupling/field_registry.hpp`) manages shared fields between physics modules:

```
class FieldRegistry {
    static FieldRegistry& instance();
    void register_field(name, provider_module, field_ptr);
    void register_consumer(name, consumer_module);
    FieldPtr get_field(name);
    void mark_synchronized(name);
};
```

Coupling Operators

The `CouplingOperator` base class (`coupling/coupling_operator.hpp`) defines the interface for field transfer between discretizations:

```
class CouplingOperator {
    virtual void initialize(const CouplingInterface& interface) = 0;
    virtual void transfer(const Field<Real>& source, Field<Real>& target) = 0;
};
```

Operator	Description
`DirectCouplingOperator`	1:1 node mapping, same discretization
`InterpolationCouplingOperator`	Weighted interpolation with neighbor weights
`NearestNeighborCouplingOperator`	Nearest node mapping

The `InterpolationCouplingOperator` uses a GPU-compatible Kokkos kernel for parallel interpolation:

```
struct InterpolationKernel {
    KOKKOS_INLINE_FUNCTION
    void operator()(const int i) const;
};
```

FEM-PD Coupling Methods

Four coupling strategies are available for FEM–PD coupling:

Arlequin Method

A blending-zone approach where FEM and PD overlap:

$$\alpha_{\text{FEM}}(\mathbf{x}) + \alpha_{\text{PD}}(\mathbf{x}) = 1 \quad \text{(partition of unity)}$$

The total energy is decomposed as:

$$E_{\text{total}} = \alpha_{\text{FEM}} E_{\text{FEM}} + \alpha_{\text{PD}} E_{\text{PD}} + E_{\text{coupling}}$$

Mortar Coupling

Segment-to-horizon coupling at the FEM–PD interface using mortar integrals for consistent force transfer.

Element Morphing

Dynamic FEM-to-PD conversion based on element damage:

1. When element damage exceeds a threshold, the element's nodes are converted to PD particles.
2. PD neighbor lists are established for the new particles.
3. State variables (stress, strain, history) are transferred from the FEM element to the PD particles.
4. The FEM element is deactivated.

Adaptive Coupling

Damage-driven zone reclassification: regions begin as FEM elements, automatically convert to PD particles when damage initiates, with dynamic blending zone management.

4.7.4 Thermal Analysis

The thermal solver (`physics/thermal_solver.hpp`) implements heat conduction with optional coupling to the mechanical solver for thermo-mechanical analysis.

Governing equation:

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) + Q$$

where ρ is the density, c_p is the specific heat capacity, k is the thermal conductivity, T is the temperature, and Q is the volumetric heat source.

Thermo-mechanical coupling: Temperature changes generate thermal strains:

$$\varepsilon_{ij}^{\text{th}} = \alpha \, \Delta T \, \delta_{ij}$$

where α is the coefficient of thermal expansion. These thermal strains are subtracted from the total strain before computing mechanical stresses.

Heat Conduction Solver

The `HeatConductionSolver` class (`physics/thermal_wave14.hpp`) implements an explicit forward-Euler scheme with a lumped capacity matrix for transient heat conduction. Nodal temperatures are updated using a connectivity-based finite-difference Laplacian stencil where each neighbor j contributes a flux $kA_{ij}(T_j - T_i)/d_{ij}$ to the lumped heat rate. The update formula per node is:

$$T_i^{n+1} = T_i^n + \frac{\Delta t}{\rho \, c_p} \left[\sum_j \frac{k \, A_{ij}}{d_{ij} \, V_i} (T_j - T_i) + Q_i \right]$$

Convection Boundary Conditions

The `ConvectionBC` class applies Newton's law of cooling on boundary faces. For each boundary node i with exposed area A, the convective heat loss is:

$$Q_{\text{conv}} = -h \, A \, (T_i - T_{\text{amb}})$$

where h is the convective heat transfer coefficient (W/m^2-K) and T_{amb} is the ambient fluid temperature. Multiple independent convection conditions (e.g., different surfaces with different h values) can be registered simultaneously.

Radiation Boundary Conditions

The `RadiationBC` class implements Stefan-Boltzmann radiation heat transfer on boundary faces. The net radiative heat loss from a node is:

$$Q_{\text{rad}} = -\sigma \, \varepsilon \, F \, A \, (T_i^4 - T_{\text{env}}^4)$$

where $\sigma = 5.670 \times 10^{-8}$ W/m^2-K^4 is the Stefan-Boltzmann constant, ε is the surface emissivity, and F is the geometric view factor. A linearized radiation coefficient $h_{\text{rad}} = 4\sigma\varepsilon T^3$ is provided for stability analysis and time-step estimation.

Fixed Temperature BC

The `FixedTemperatureBC` class enforces Dirichlet temperature constraints by overwriting nodal temperatures to prescribed values after each solver step. It is applied last among all boundary conditions so that the fixed constraint takes precedence. Multiple node groups with different prescribed temperatures can be registered.

Heat Flux BC

The `HeatFluxBC` class applies Neumann (prescribed flux) boundary conditions on boundary faces. For each boundary node with area A, the heat input is $Q = q\,A$ where q (W/m^2) is the prescribed flux, positive into the domain. Per-node or uniform boundary areas are supported.

Adiabatic Heating

The `AdiabaticHeating` class computes the Taylor-Quinney temperature rise from plastic dissipation in high-rate deformation:

$$\Delta T = \frac{\eta\,\sigma_{\text{eq}}\,\Delta\varepsilon^p}{\rho\,c_p}$$

where η is the Taylor-Quinney coefficient (default 0.9 for metals), σ_{eq} is the von Mises stress, and $\Delta\varepsilon^p$ is the plastic strain increment. A power-based variant accepting plastic power $\dot{W}^p = \sigma : \dot{\varepsilon}^p$ over a time step is also available.

Thermal Time Step Control

The `ThermalTimeStep` class computes the stable time step for the explicit forward-Euler thermal scheme:

$$\Delta t_{\text{therm}} = \frac{h^2\,\rho\,c_p}{2\,k}$$

where h is the minimum element characteristic length. The thermal time step is typically much larger than the mechanical CFL limit, so the class also computes the recommended number of thermal subcycles per mechanical step via $N_{\text{sub}} = \lceil \Delta t_{\text{mech}} / \Delta t_{\text{therm}} \rceil$.

Coupled Thermo-Mechanical

The `CoupledThermoMechanical` class manages staggered (isothermal-split) coupling between the mechanical and thermal solvers. Each coupled time step follows the sequence:

1. **Mechanical step** with the current temperature field.
2. **Adiabatic heating** from plastic work is buffered and added to nodal temperatures.
3. **Thermal sub-stepping** with N_{sub} forward-Euler steps, applying conduction, convection, radiation, heat flux, and fixed-temperature BCs at each sub-step.
4. **Material property update** from the new temperature field (e.g., temperature-dependent yield stress or thermal conductivity).

Energy conservation is monitored via a per-step energy balance diagnostic.

4.8 Part VIII — Advanced Features

This part describes the ALE mesh management module, the sensor and control subsystems, the advanced capabilities (Wave 16), and the parallel computing framework (Wave 17).

4.8.1 ALE Mesh Management

The Arbitrary Lagrangian–Eulerian (ALE) module (`physics/ale_solver.hpp`) combines the advantages of the Lagrangian and Eulerian descriptions of motion. In the Lagrangian phase, the mesh deforms with the material; in the ALE phase, the mesh is smoothed to maintain element quality, and the state variables are advected (remapped) to the new mesh positions.

ALE Process

At each time step, the ALE solver performs three operations in sequence:

1. **Lagrangian step** — The standard FEM explicit or implicit update is applied, deforming the mesh with the material.
2. **Mesh smoothing** — Node positions are relocated to improve element quality while maintaining the domain boundary.
3. **Advection** — State variables (density, velocity, energy) are remapped from the Lagrangian mesh to the smoothed mesh.

Smoothing Methods

Method	Algorithm
Laplacian	$\mathbf{x}_{\text{new}} = (1 - w)\mathbf{x} + w\,\overline{\mathbf{x}}_{\text{neighbors}}$
Weighted	Distance-weighted Laplacian (closer neighbors have more influence)
Equipotential	Solves the Laplace equation on the mesh (falls back to Laplacian)

The smoothing weight $w \in [0, 1]$ controls the strength of the smoothing. Boundary nodes are held fixed unless boundary smoothing is explicitly enabled.

Advection Methods

Method	Order	Properties
Donor Cell	1st	Upwind scheme; simple, robust, diffusive
Van Leer	2nd	Minmod limiter; less diffusive, TVD-stable

API Reference

```
class ALESolver {
public:
    // Smoothing
    Real smooth(Real* coordinates);

    // Advection
    void advect_scalar(const Real* old_coords, const Real* new_coords,
                       Real* field, Real dt);
```

(continues on next page)

(continued from previous page)

```
    void advect_vector(const Real* old_coords, const Real* new_coords,
                       Real* field, Real dt);

    // Combined ALE step
    Real ale_step(Real* coords, Real* velocities,
                  const std::vector<Real*>& scalars, Real dt);

    // Element quality metrics
    static Real element_quality(const Real* coords, const Index* nodes, int npe);
    Real average_quality(const Real* coords, const Index* conn,
                         std::size_t ne, int npe) const;
};
```

The `element_quality()` method returns a value in $[0, 1]$, where 1 represents a perfectly regular element (cube for hexahedra, equilateral tetrahedron for tets). The `ale_step()` method returns the average element quality after smoothing.

4.8.2 Sensors and Controls

The sensor and control subsystem enables runtime monitoring of simulation quantities and automated response to user-defined conditions.

Sensor Types

Five sensor types are provided (`fem/sensor.hpp`):

Sensor	Measurement	Parameters
Accelerometer	Nodal acceleration (x, y, or z)	`node_id`, `component`
Velocity Gauge	Nodal velocity	`node_id`, `component`
Distance Sensor	Distance between two nodes	`node_a`, `node_b`
Strain Gauge	Element strain component	`elem_id`, `component`
Force Sensor	Reaction force at constrained node	`node_id`, `dof`

Sensor API

```
class Sensor {
    void sample(const State& state, Real time);
    Real current_value() const;
    const std::vector<std::pair<Real,Real>>& history() const;

    // CFC filtering (SAE J211 standard)
    void enable_cfc_filter(int cfc_class);
    Real filtered_value() const;
};
```

(continues on next page)

(continued from previous page)

```
class SensorManager {
    Index add_sensor(const Sensor& sensor);
    void sample_all(const State& state, Real time);
    void set_sampling_interval(Real dt);
};
```

CFC Filtering

The Channel Frequency Class (CFC) filter implements the SAE J211 standard for filtering crash test data. Supported classes:

CFC Class	Cutoff Frequency	Application
CFC 60	100 Hz	Structural deformation
CFC 180	300 Hz	Vehicle accelerations
CFC 600	1,000 Hz	Head/chest accelerations
CFC 1000	1,650 Hz	Helmet/component response

Control System

The `ControlAction` class (`fem/controls.hpp`) defines condition–action pairs that execute when a sensor reading crosses a threshold:

```
class ControlAction {
    void set_sensor(Index sensor_id);
    void set_threshold(Real value);
    void set_action(ActionType type);
    void set_mode(ControlMode mode);    // OneShot or Repeating
    bool check_and_execute(const SensorManager& sensors,
                           State& state, Real time);
};
```

Action Types

Action Type	Description
Terminate	End the simulation
Activate Load	Turn on a load
Deactivate Load	Turn off a load
Activate Contact	Enable a contact interface
Deactivate Contact	Disable a contact interface
Activate Failure	Enable failure checking
Deactivate Failure	Disable failure checking
Change Timestep	Modify the time step
Write Checkpoint	Trigger a checkpoint write
Write Animation	Trigger an animation frame
Custom Callback	Execute a user-defined `std::function`

4.8.3 Advanced Capabilities (Wave 16)

The advanced capabilities module (`physics/advanced_wave16.hpp`) provides eight sub-modules for specialized simulation scenarios.

Modal Analysis (Lanczos Eigensolver)

The `LanczosEigensolver` class solves the generalized eigenvalue problem $\mathbf{Kx} = \lambda \mathbf{Mx}$ for the lowest N eigenpairs using inverse iteration with M-orthogonal deflation. A CG inner solver handles the linear system at each iteration, falling back to direct Gaussian elimination for systems with $n \leq 64$. Shift-invert mode (σ-shift) is supported for extracting interior eigenvalues. Natural frequencies are obtained as $f_i = \sqrt{\lambda_i}/(2\pi)$.

XFEM

The Extended Finite Element Method module models cracks without remeshing. The `LevelSetCrack` class represents the crack geometry using two signed-distance fields: ϕ (normal to the crack surface) and ψ (tangential, ahead of the crack tip). The `XFEMEnrichment` class provides Heaviside enrichment $H(\phi)$ for fully cut elements and four asymptotic branch functions $\sqrt{r}\{\sin\frac{\theta}{2}, \cos\frac{\theta}{2}, \sin\frac{\theta}{2}\sin\theta, \cos\frac{\theta}{2}\sin\theta\}$ for tip elements. Crack propagation uses the maximum hoop stress criterion (Erdogan-Sih) with J-integral evaluation.

CONWEP Blast Loading

The `CONWEPBlast` class implements the Friedlander waveform for blast pressure-time history:

$$P(t) = P_s \left(1 - \frac{t}{t_{\text{pos}}}\right) \exp\left(-b\,\frac{t}{t_{\text{pos}}}\right)$$

Peak overpressure and positive-phase duration are computed from Kingery-Bulmash empirical fits using Hopkinson-Cranz scaling $Z = R/W^{1/3}$. Oblique reflection coefficients account for angle of incidence.

Airbag Simulation

The `AirbagModel` class simulates gas-inflated airbag deployment using the ideal gas law with mass inflow, vent holes, and fabric porosity. Mass conservation tracks inflator inflow, orifice-based vent discharge (subsonic and choked flow regimes), and Darcy-law fabric leakage. The energy balance couples pressure work $P\,dV$, inflator enthalpy input, and outflow losses to update gas temperature and pressure each time step.

Seatbelt Dynamics

The `BeltElement` class models belt webbing as a 1D tension-only bar element with nonlinear stiffness, rate-dependent damping, and slip-ring feed-through using Euler's belt friction formula $T_{\text{tight}}/T_{\text{slack}} = e^{\mu\theta}$. The `Retractor` class adds spool-out, deceleration-triggered locking, pyrotechnic pretensioner (timed force retraction), and a force-limiting load limiter that caps chest loading above a specified threshold.

Advanced ALE

The `EulerianSolver` class solves the 1D Euler equations on a fixed Cartesian grid using first-order upwind (donor-cell) advection in conservation form for density, momentum, and total energy. The `CutCellMethod` class tracks multi-material volume fractions (up to four materials) with PLIC interface reconstruction and small-cell stabilization. The `TurbulenceModel` class implements the standard k-ε model (Launder-Sharma constants) with wall functions for the log-law region.

Adaptive Mesh Refinement

The `AMRManager` class implements h-refinement driven by the Zienkiewicz-Zhu (ZZ) superconvergent patch recovery error estimator. Element errors are computed by comparing FE stresses against neighbor-averaged recovered stresses. Elements are marked for refinement when their error exceeds a fraction of the maximum, and for coarsening when below a lower threshold. Quadrilateral elements are subdivided into four children, with hanging-node constraints ($u_{\text{hang}} = \frac{1}{2}(u_L + u_R)$) enforced at non-conforming edges.

Draping Analysis

The `DrapingAnalysis` class performs kinematic fiber draping of composite fabrics onto curved mold surfaces using the fishnet algorithm (Mack-Taylor). Starting from a pin point, geodesic paths along warp and weft directions are propagated across the surface mesh. At each cell the shear angle $\gamma = \pi/2 - \alpha$ (deviation from right-angle fiber intersection) is computed from edge vectors. Elements exceeding a user-specified lock angle are flagged as manufacturing-infeasible.

4.8.4 Parallel Computing (Wave 17)

The parallel computing module (`parallel/mpi_wave17.hpp`) provides six MPI-based components for distributed simulation. All MPI calls are guarded by `#ifdef NEXUSSIM_HAVE_MPI` with serial fallbacks for single-rank execution.

Distributed Stiffness Assembly

The `DistributedAssembler` class manages per-rank assembly of the global stiffness matrix in CSR format. Each rank owns a contiguous range of global rows and accumulates element contributions as triplets (COO format), which are converted to CSR via `build_csr_from_triplets()` with in-row column sorting and duplicate merging. Ghost node force contributions are buffered separately and exchanged with owning ranks.

Ghost Node Communication

The `GhostExchanger` class handles asynchronous field synchronization across partition boundaries. Communication patterns are set up once from ghost-owner maps, then reused each step via `begin_exchange()` / `finish_exchange()` (non-blocking `MPI_Isend` / `MPI_Irecv`). Multi-DOF fields are supported through a configurable `dofs_per_node` parameter that packs and unpacks vector-valued data (displacements, velocities, etc.).

Domain Decomposition

The `DomainDecomposer` class provides two partitioning strategies. Recursive Coordinate Bisection (RCB) splits along the longest dimension at weighted-median split points, recursing until the target number of partitions is reached. The weighted greedy method assigns elements to partitions using a score that balances load (lowest current weight) against communication (most neighbors already assigned). Partition quality is reported as load imbalance ratio and edge-cut count.

Parallel Contact Detection

The `ParallelContactDetector` class implements a two-phase contact search. In the broad phase, per-rank AABBs for contact surfaces A and B are exchanged to identify overlapping rank pairs. In the narrow phase, element-level AABB intersection tests (inflated by a search tolerance) produce `ContactPair` records containing surface IDs, owning ranks, estimated gap distance, and approximate contact point coordinates.

Load Balancing

The `LoadBalancer` class monitors per-rank computational weights and generates migration plans when the imbalance ratio (max/average weight) exceeds a configurable threshold. The greedy diffusion algorithm moves elements from overloaded ranks to the most underloaded rank, subject to the constraint that transfers must strictly improve balance. The resulting `MigrationPlan` records each element move with source rank, destination rank, and element weight.

Scalability Benchmarking

The `ScalabilityBenchmark` class provides timing infrastructure for parallel performance analysis. Named phases (assembly, communication, solve) are timed individually via `start_phase()` / `stop_phase()`. After collecting results across multiple rank counts, `compute_scaling_metrics()` derives speedup $S(n) = T(1)/T(n)$, parallel efficiency $E(n) = S(n)/n$, and communication-to-computation ratio. A formatted report is generated via `report()`.

4.9 Part IX — Verification and Validation

4.9.1 Test Suite

The NexusSim test suite comprises 95 test executables containing over 2,100 individual assertions. Tests are organized as standalone C++ programs that use a lightweight assertion framework.

Test Infrastructure

Tests use a custom `CHECK` macro that reports failures with file and line information:

```
#define CHECK(cond, msg) \
    if (!(cond)) { \
        std::cerr << "FAIL: " << msg << " at " \
                  << __FILE__ << ":" << __LINE__ << "\n"; \
        return 1; \
    }
```

Each test program follows the pattern:

```
int main() {
    int passed = 0, failed = 0;

    // Test case 1
    {
        // Setup, execute, assert
        CHECK(result == expected, "description");
        passed++;
    }

    std::cout << passed << "/" << (passed + failed) << " passed\n";
    return (failed > 0) ? 1 : 0;
}
```

Test Summary by Module

Test File	Assertions	Coverage Area
`lsdyna_reader_ext_test.cpp`	172	LS-DYNA reader extensions
`sensor_ale_test.cpp`	104	Sensors (5 types), controls (8 actions), ALE
`restart_output_test.cpp`	110	Checkpoint/restart, time history
`enhanced_output_test.cpp`	97	Extended output modules
`pd_enhanced_test.cpp`	99	PD correspondence, bonds, morphing

continues on next page

Table 4.1 – continued from previous page

Test File	Assertions	Coverage Area
composite_layup_test.cpp	83	CLT, ABD matrix, section properties
composite_progressive_test cpp	78	Thermal stress, progressive failure
realistic_crash_test.cpp	63	14 multi-system integration scenarios
material_models_test.cpp	62	All 14 material models
failure_models_test.cpp	53	All 6 failure models
hertzian_mortar_test.cpp	53	Hertzian + mortar contact
rigid_body_test.cpp	51	Rigid bodies + constraints
implicit_validation_test. cpp	46	Implicit solver multi-element validation
loads_system_test.cpp	47	Load curves + initial conditions
fem_robustness_test.cpp	36	Solver robustness guards
tied_contact_eos_test.cpp	35	Tied contact + 5 EOS models
fem_pd_integration_test. cpp	29	FEM-PD integration (Arlequin)
pd_fem_coupling_test.cpp	39	FEM-PD coupling
arc_length_test.cpp	25	Arc-length (snap-through, truss, arch)
shell4_solver_test.cpp	24	Shell4 6-DOF solver integration
mpi_partition_test.cpp	23	MPI partitioning
explicit_dynamics_test. cpp	114	Explicit solver, bulk viscosity, hourglass, energy, erosion
material_wave10_test.cpp	128	20 material models (Hill, Barlat, concrete, fabric, etc.)
failure_wave11_test.cpp	101	12 failure models (J-C, Lemaitre, Puck, FLD, etc.)
contact_wave12_test.cpp	60	Contact expansion (self-contact, edge-to-edge, etc.)
eos_wave13_test.cpp	49	8 EOS models (Murnaghan, Tillotson, Sesame, etc.)
thermal_wave14_test.cpp	49	Thermal solver (conduction, convection, radiation, coupling)
elements_wave15_test.cpp	80	7 element formulations (thick shell, DKT/DKQ, plane, axi)
advanced_wave16_test.cpp	77	8 advanced capabilities (modal, XFEM, blast, airbag, etc.)
mpi_wave17_test.cpp	82	MPI completion (assembly, ghost exchange, decomposition)

Test Categories

Unit tests. Individual material models, element formulations, and solver components are tested in isolation. For example, `material_models_test.cpp` verifies the stress response of each of the 14 material models under prescribed strain paths.

Integration tests. Multi-system scenarios test the interaction between materials, contact, loads, and the solver. The `realistic_crash_test.cpp` file contains 14 integration scenarios modeling vehicle crash components.

Regression tests. Known analytical solutions are used to verify numerical accuracy. Examples include cantilever beam bending (compared to Euler–Bernoulli beam theory) and patch tests (uniform strain should produce zero internal force error).

Robustness tests. Degenerate inputs are tested to ensure graceful failure: zero-volume elements, NaN propagation, singular stiffness matrices, and unconstrained systems. The `fem_robustness_test.cpp` validates all diagnostic checks.

Convergence tests. Mesh refinement studies verify optimal convergence rates: linear elements should exhibit $O(h)$ error in energy norm, and quadratic elements should exhibit $O(h^2)$.

Running the Test Suite

```
# Build all targets (includes tests)
cmake --build build -j$(nproc)

# Run a single test
./build/bin/material_models_test

# Run all tests with summary
passed=0; failed=0
for test in build/bin/*_test; do
    if "$test" > /dev/null 2>&1; then
        ((passed++))
    else
        echo "FAILED: $test"
        ((failed++))
    fi
done
echo "$passed passed, $failed failed"
```

Known Limitations

1. **Shell4 shear locking.** Full-integration Shell4 elements exhibit shear locking for thin shells, producing deflections approximately 1.5% of Euler–Bernoulli beam theory. This is a known element limitation, not a solver defect.

2. **GPU execution.** The CUDA build has been validated on an NVIDIA GeForce GT 1030 (Pascal, compute capability 6.1, 2 GB GDDR5, 384 CUDA cores) using the project-local Kokkos CUDA installation (`external/kokkos/`). All 35 GPU sparse solver tests pass on the CUDA backend with

correctness matching the CPU to machine precision. See *Chapter 2* (page 7) for GPU build instructions and the GPU Benchmark Results section below for performance data.

3. **MPI integration.** The MPI infrastructure includes distributed assembly, ghost exchange, domain decomposition (RCB), parallel contact detection, and load balancing (Wave 17). Full production-scale validation on multi-node clusters is pending.

GPU Benchmark Results

The following benchmarks were collected on the development system comparing the OpenMP backend (64 threads) against the CUDA backend (NVIDIA GeForce GT 1030, 384 CUDA cores, 2 GB GDDR5). The GT 1030 is an entry-level desktop GPU; production compute GPUs (e.g., A100, H100) would show substantially larger speedups.

Kokkos Micro-Benchmarks (`kokkos_performance_test`)

Operation	Size	CPU (ms)	GPU (ms)	Speedup
Vector Add (DAXPY)	10K	6.39	0.07	96.8x
Dot Product	10K	5.44	0.13	40.9x
SpMV (7 nnz/row)	10K	4.86	0.14	35.0x
Element Forces	1.25K	9.40	0.14	68.6x
Time Integration	30K	4.47	0.14	31.7x
Vector Add (DAXPY)	100K	4.32	0.19	22.5x
Dot Product	100K	4.49	0.33	13.8x
SpMV (7 nnz/row)	100K	4.43	1.83	2.4x
Element Forces	12.5K	9.08	0.18	49.6x
Time Integration	300K	4.46	1.42	3.1x
Vector Add (DAXPY)	1M	4.57	2.03	2.2x
Dot Product	1M	4.41	1.53	2.9x
SpMV (7 nnz/row)	1M	7.02	14.25	0.49x
Element Forces	125K	10.31	1.53	6.7x
Time Integration	3M	11.23	14.39	0.78x

At small-to-medium sizes (10K–100K), the GPU delivers 3x–97x speedup. At 1M elements, memory-bound operations (SpMV, time integration) become bandwidth-limited on the GT 1030's 64-bit memory bus, and the 64-thread CPU prevails. Element force computation remains GPU-favorable (6.7x at 125K elements) because it is compute-bound.

Sparse Solver (`gpu_sparse_test`)

Metric	CPU (OpenMP)	GPU (CUDA)	Speedup
SpMV (flat), 27K DOFs	4.47 ms	0.58 ms	7.7x
SpMV (team), 27K DOFs	66.40 ms	0.95 ms	70.2x
CG Solve, 27K DOFs, 76 iter	2955.7 ms	118.5 ms	24.9x
CG per iteration	38.89 ms	1.56 ms	24.9x

All 35 correctness tests pass on the CUDA backend. CPU and GPU CG solutions match to machine precision ($\varepsilon < 10^{-15}$).

FEM Hex8 Explicit Dynamics (`gpu_performance_benchmark`)

Elements	DOFs	CPU (ms/step)	GPU (ms/step)	Speedup
125	216	13.10	1.68	7.8x
1,000	726	12.46	1.61	7.7x
8,000	3,969	11.47	1.99	5.8x
27,000	11,532	13.53	2.72	5.0x

The GPU achieves 5–8x speedup at these moderate problem sizes. At smaller sizes, GPU launch overhead dominates; at larger sizes the GT 1030's memory bandwidth limits scaling.

Peridynamics (`pd_large_benchmark`)

Particles	Bonds	CPU Force (ms)	GPU Force (ms)	CPU M bonds/s	GPU M bonds/s
27,000	2.9M	16.2	77.3	181.3	37.9
50,653	5.6M	19.9	187.6	282.8	30.0
97,336	11.0M	35.2	350.3	312.9	31.4
125,000	14.2M	26.6	442.5	534.4	32.1

The 64-thread CPU outperforms the GT 1030 for peridynamics force computation. This is expected: the PD bond-force kernel performs gather-scatter operations that stress the GT 1030's limited memory bandwidth (48 GB/s) versus the CPU's aggregate memory bandwidth (~100+ GB/s across 64 threads). A production GPU (e.g., NVIDIA A100 with 2 TB/s bandwidth) would reverse this ratio by approximately 40x.

Peak Throughput Summary

Backend	Element Processing	DOF Update Rate
CPU (OpenMP)	1.21×10^7 elem/s	2.67×10^8 DOF/s
GPU (CUDA)	8.16×10^7 elem/s	2.13×10^8 DOF/s

The GPU achieves 6.7x higher peak element processing throughput. DOF update rate is comparable because the time integration kernel is memory-bandwidth-limited on the GT 1030.

4.10 Appendices

4.10.1 Appendix A — Complete Header File Index

The following is a complete listing of all 126 public header files, organized by directory.

core/ (7 files)

File	Purpose
`core.hpp`	Aggregate include for all core headers
`types.hpp`	Fundamental types, vectors, enumerations, constants
`exception.hpp`	Exception hierarchy and assertion macros
`gpu.hpp`	Kokkos abstraction layer, parallel wrappers
`logger.hpp`	Logging singleton and macros
`memory.hpp`	Memory arena, pool, aligned buffer, tracker
`mpi.hpp`	MPI manager singleton

data/ (4 files)

File	Purpose
`data.hpp`	Aggregate include for all data headers
`field.hpp`	Generic SOA field container
`mesh.hpp`	Mesh: nodes, element blocks, node/side sets
`state.hpp`	State, StateHistory

coupling/ (3 files)

File	Purpose
`coupling.hpp`	Aggregate include
`coupling_operator.hpp`	Coupling operator base and implementations
`field_registry.hpp`	Shared field registry singleton

discretization/ (11 files)

File	Purpose
`hex8.hpp`	8-node hexahedron element
`hex20.hpp`	20-node serendipity hexahedron
`tet4.hpp`	4-node tetrahedron
`tet10.hpp`	10-node quadratic tetrahedron
`wedge6.hpp`	6-node wedge/prism
`shell3.hpp`	3-node triangular shell
`shell4.hpp`	4-node quadrilateral shell (6-DOF)
`beam2.hpp`	2-node beam element
`truss.hpp`	2-node truss element
`spring_damper.hpp`	Spring, damper, spring-damper elements
`mesh_partition.hpp`	Mesh partitioning for MPI

solver/ (6 files)

File	Purpose
`implicit_solver.hpp`	Newton–Raphson solver
`fem_static_solver.hpp`	FEM static solver (linear + arc-length)
`arc_length_solver.hpp`	Crisfield's cylindrical arc-length
`petsc_solver.hpp`	PETSc linear solver integration
`sparse_matrix.hpp`	CSR sparse matrix
`gpu_sparse_matrix.hpp`	GPU-compatible sparse matrix

physics/ (29 files)

File	Purpose
`element.hpp`	Element base class and factory
`element_erosion.hpp`	Element erosion criteria
`material.hpp`	MaterialState, MaterialProperties, MaterialType enum
`material_models.hpp`	Aggregate: elastic, Von Mises, Johnson-Cook, Neo-Hookean
`material_library.hpp`	Predefined material presets
`material_orthotropic.hpp`	Orthotropic elastic
`material_mooney_rivlin.hpp`	Mooney-Rivlin hyperelastic
`material_ogden.hpp`	Ogden hyperelastic
`material_piecewise_linear.hpp`	Piecewise-linear plasticity
`material_tabulated.hpp`	Tabulated stress-strain
`material_foam.hpp`	Low-density foam
`material_crushable_foam.hpp`	Crushable foam
`material_honeycomb.hpp`	Honeycomb orthotropic crush
`material_viscoelastic.hpp`	Viscoelastic (Prony series)
`material_cowper_symonds.hpp`	Cowper-Symonds rate-dependent
`material_zhao.hpp`	Zhao rate-dependent
`material_elastic_plastic_fail.hpp`	Elastic-plastic with failure
`material_rigid.hpp`	Rigid material
`material_null.hpp`	Null (mass-only) material
`eos.hpp`	Equations of state (5 models)
`section.hpp`	Section properties
`large_deformation.hpp`	Large deformation utilities
`time_integration.hpp`	Time integration schemes
`time_integrator.hpp`	Time integrator interface
`adaptive_timestep.hpp`	Adaptive time step controller
`ale_solver.hpp`	ALE mesh management
`thermal_solver.hpp`	Thermal/thermo-mechanical solver
`module.hpp`	Physics module interface
`physics.hpp`	Aggregate include

physics/failure/ (8 files)

File	Purpose
`failure_model.hpp`	FailureState, FailureModel base class
`failure_models.hpp`	Aggregate include for all failure models
`failure_hashin.hpp`	Hashin 4-mode composite failure
`failure_tsai_wu.hpp`	Tsai-Wu quadratic interaction
`failure_chang_chang.hpp`	Chang-Chang modified Hashin
`failure_gtn.hpp`	GTN ductile void growth
`failure_gissmo.hpp`	GISSMO incremental damage
`failure_tabulated.hpp`	Tabulated failure strain

physics/composite/ (5 files)

File	Purpose
`composite_layup.hpp`	Ply definition, CompositeLaminate, ABD matrix
`composite_thermal.hpp`	Thermal residual stress
`composite_interlaminar.hpp`	Interlaminar shear stress
`composite_progressive_failure.hpp`	FPF, ply degradation, strength envelope
`composite_utils.hpp`	Laminate utilities

fem/ (18 files)

File	Purpose
`fem_solver.hpp`	Explicit FEM solver
`contact.hpp`	Penalty contact
`edge_contact.hpp`	Edge-based contact
`surface_contact.hpp`	Surface-to-surface contact
`node_to_surface_contact.hpp`	Node-to-surface projection
`hertzian_contact.hpp`	Hertzian sphere contact
`mortar_contact.hpp`	Mortar segment-to-segment
`tied_contact.hpp`	Tied contact with failure
`voxel_collision.hpp`	Voxel-based collision detection
`friction_model.hpp`	Coulomb friction with decay
`rigid_body.hpp`	Rigid body dynamics
`rigid_wall.hpp`	Rigid wall constraints
`constraints.hpp`	RBE2, RBE3, joints
`load_curve.hpp`	Load curve and manager
`loads.hpp`	Load manager
`initial_conditions.hpp`	Initial velocity, temperature
`sensor.hpp`	Sensor types and manager
`controls.hpp`	Control actions and logic

io/ (14 files)

File	Purpose
`mesh_reader.hpp`	Generic mesh reader interface
`mesh_validator.hpp`	Mesh quality validation
`lsdyna_reader.hpp`	LS-DYNA keyword file reader
`radioss_reader.hpp`	Radioss Starter deck reader
`config_reader.hpp`	YAML configuration parser
`vtk_writer.hpp`	VTK output (legacy + XML)
`checkpoint.hpp`	Basic binary checkpoint
`extended_checkpoint.hpp`	Extended checkpoint (history, contact)
`time_history.hpp`	Time history output
`animation_writer.hpp`	Animation sequence writer
`part_energy.hpp`	Part energy output
`interface_force_output.hpp`	Interface force output
`cross_section_force.hpp`	Cross-section force/moment output
`result_database.hpp`	Binary result database

sph/ (4 files)

File	Purpose
`sph_kernel.hpp`	SPH kernel functions
`sph_solver.hpp`	WCSPH solver
`neighbor_search.hpp`	Spatial hashing neighbor search
`fem_sph_coupling.hpp`	FEM-SPH coupling

peridynamics/ (15 files)

File	Purpose
`pd_types.hpp`	PD Kokkos view type aliases
`pd_particle.hpp`	PDParticles data structure
`pd_neighbor.hpp`	PDNeighborList (CSR format)
`pd_force.hpp`	Bond-based force computation
`pd_solver.hpp`	PD time integration solver
`pd_state_based.hpp`	Ordinary state-based PD
`pd_correspondence.hpp`	Non-ordinary correspondence PD
`pd_bond_models.hpp`	Enhanced bond models
`pd_materials.hpp`	JC, Drucker-Prager, JH2 materials
`pd_morphing.hpp`	FEM-to-PD element morphing
`pd_mortar_coupling.hpp`	Mortar FEM-PD coupling
`pd_adaptive_coupling.hpp`	Damage-driven adaptive coupling
`pd_fem_coupling.hpp`	General FEM-PD coupling interface
`pd_contact.hpp`	PD-based contact
`peridynamics.hpp`	Aggregate include

utils/ (1 file)

File	Purpose
`performance_timer.hpp`	Timer, ScopedTimer, Profiler, BenchmarkResult

4.10.2 Appendix B — Mathematical Notation

Tensor Notation

Symbol	Description
$\boldsymbol{\sigma}$	Cauchy stress tensor
$\boldsymbol{\varepsilon}$	Infinitesimal strain tensor
$\mathbf{F}$	Deformation gradient: $F_{ij} = \partial x_i / \partial X_j$
$\mathbf{B}$	Left Cauchy–Green: $\mathbf{B} = \mathbf{F}\mathbf{F}^T$
$\mathbf{C}$	Right Cauchy–Green: $\mathbf{C} = \mathbf{F}^T\mathbf{F}$
J	Jacobian determinant: $J = \det(\mathbf{F})$
$\mathbf{I}$	Second-order identity tensor
δ_{ij}	Kronecker delta
ϵ_{ijk}	Levi-Civita permutation symbol
$\mathbf{D}$	Rate of deformation tensor
$\mathbf{W}$	Spin (vorticity) tensor

Voigt Convention

The Voigt notation maps symmetric tensors to vectors:

$$\begin{bmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} \\ \sigma_{12} & \sigma_{22} & \sigma_{23} \\ \sigma_{13} & \sigma_{23} & \sigma_{33} \end{bmatrix} \longrightarrow \begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \\ \sigma_4 \\ \sigma_5 \\ \sigma_6 \end{bmatrix} = \begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{zz} \\ \sigma_{xy} \\ \sigma_{yz} \\ \sigma_{xz} \end{bmatrix}$$

Engineering shear strains: $\gamma_{ij} = 2\varepsilon_{ij}$ for $i \neq j$.

Continuum Mechanics

Strong form (balance of linear momentum):

$$\rho\,\ddot{\mathbf{u}} = \nabla \cdot \boldsymbol{\sigma} + \mathbf{b} \quad \text{in } \Omega$$

$$\mathbf{u} = \bar{\mathbf{u}} \quad \text{on } \Gamma_D \qquad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{on } \Gamma_N$$

Weak form (principle of virtual work):

$$\int_\Omega \boldsymbol{\sigma} : \delta\boldsymbol{\varepsilon}\,\mathrm{d}V = \int_\Omega \mathbf{b} \cdot \delta\mathbf{u}\,\mathrm{d}V + \int_{\Gamma_N} \bar{\mathbf{t}} \cdot \delta\mathbf{u}\,\mathrm{d}A$$

Discretized form:

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{F}_{\text{ext}}$$

Lamé Parameters

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \qquad \mu = G = \frac{E}{2(1+\nu)}$$

Bulk and Shear Moduli

$$K = \frac{E}{3(1-2\nu)}, \qquad G = \frac{E}{2(1+\nu)}$$

4.10.3 Appendix C — Material Parameter Reference

Elastic Materials

Material	E (GPa)	ν	ρ (kg/m^3)	σ_y (MPa)
Mild Steel	210	0.30	7,850	250
Stainless Steel 304	193	0.29	8,000	215
Aluminum 6061-T6	69	0.33	2,700	276
Aluminum 7075-T6	72	0.33	2,810	503
Titanium Ti-6Al-4V	114	0.34	4,430	880
Copper OFHC	117	0.34	8,960	33
Concrete	30	0.20	2,400	—
Glass (Soda-Lime)	72	0.22	2,500	—

Hyperelastic Materials

Material	C_{10} (MPa)	C_{01} (MPa)	D_1 (1/MPa)	ρ (kg/m^3)
Natural Rubber	0.16	0.04	0.005	1,100
Silicone Rubber	0.12	0.03	0.01	1,200
Neoprene	0.30	0.08	0.003	1,230

Foam Materials

Material	E (MPa)	ν	ρ (kg/m^3)	σ_y (MPa)	ε_d
EPS Foam	5	0.10	30	0.1	0.85
PU Foam	20	0.15	60	0.4	0.80
Aluminum Foam	1,000	0.30	300	2.0	0.70

Johnson-Cook Parameters

Material	A (MPa)	B (MPa)	n	C	m
Steel 4340	792	510	0.26	0.014	1.03
Al 7075-T6	520	477	0.52	0.001	1.61
Ti-6Al-4V	1,098	1,092	0.93	0.014	1.10
OFHC Copper	90	292	0.31	0.025	1.09

4.10.4 Appendix D — Element Reference Cards

Hex8 Reference Card

Property	Value
Type	8-node hexahedron
Nodes	8 (corner)
DOFs per node	3 (translation)
Total DOFs	24
Shape functions	Trilinear
Integration	$2 \times 2 \times 2$ = 8 Gauss points
Topology	Hexahedron
Known issues	Shear locking in bending
Recommended for	Regular geometry, moderate deformation

Hex20 Reference Card

Property	Value
Type	20-node serendipity hexahedron
Nodes	20 (8 corner + 12 mid-edge)
DOFs per node	3 (translation)
Total DOFs	60
Shape functions	Serendipity quadratic
Integration	$3 \times 3 \times 3$ = 27 Gauss points
Known issues	High computational cost
Recommended for	Stress concentrations, curved geometry

Tet4 Reference Card

Property	Value
Type	4-node tetrahedron
Nodes	4 (corner)
DOFs per node	3 (translation)
Total DOFs	12
Shape functions	Linear (volume coordinates)
Integration	1 Gauss point (centroid)
Known issues	Constant strain, volumetric locking
Recommended for	Complex geometry, automatic meshing

Tet10 Reference Card

Property	Value
Type	10-node quadratic tetrahedron
Nodes	10 (4 corner + 6 mid-edge)
DOFs per node	3 (translation)
Total DOFs	30
Shape functions	Quadratic
Integration	4 Gauss points
Recommended for	Accurate results with tet meshes

Shell4 Reference Card

Property	Value
Type	4-node quadrilateral shell
Nodes	4 (corner)
DOFs per node	6 (3 translation + 3 rotation)
Total DOFs	24
Shape functions	Bilinear
Integration	$2 \times 2 = 4$ Gauss points
Theory	Reissner–Mindlin (thick shell)
Known issues	Shear locking (~1.5% of beam theory)
Recommended for	Thin-walled structures, vehicle panels

Beam2 Reference Card

Property	Value
Type	2-node beam
Nodes	2
DOFs per node	6 (3 translation + 3 rotation)
Total DOFs	12
Shape functions	Linear (axial) / Hermite cubic (bending)
Integration	2 Gauss points
Recommended for	Frame structures, reinforcement

Truss Reference Card

Property	Value
Type	2-node truss
Nodes	2
DOFs per node	3 (translation)
Total DOFs	6
Shape functions	Linear
Integration	1 Gauss point
Recommended for	Cable structures, bracing